

Greenplum

Technical Publications

# Greenplum Performance Monitor Guide

Last Revised: December 9, 2008 11:20 am

# Contents - Greenplum Performance Monitor Guide


About This Guide ... 1
Greenplum Performance Monitor Documentation ... 1
Document Conventions ... 1
Contact Us ... 4

**Chapter 1: About Greenplum Performance Monitor** ... 5
Greenplum Performance Monitor Architecture ... 5
Greenplum Performance Monitor Agents ... 6
Greenplum Performance Monitor Database ... 6
Current Data Tables ... 7
Tail Data Tables ... 7
Historical Data Tables ... 7
Greenplum Performance Monitor Console ... 7
Greenplum Performance Monitor Web Service ... 8

**Chapter 2: Installing Greenplum Performance Monitor** ... 9
Preparing for Installation ... 9
Creating the Monitor Superuser ... 9
Editing postgresql.conf ... 9
System Requirements ... 10
Installing Greenplum Performance Monitor on the Master ... 10
About Your Greenplum Performance Monitor Installation ... 11
Installing Greenplum Performance Monitor on the Master ... 11
Setting up Greenplum Performance Monitor on the Master ... 12
Installing the Segment Monitor Agents ... 14
Next Steps ... 15
Uninstalling Greenplum Performance Monitor ... 15

**Chapter 3: Administering Greenplum Performance Monitor** 17
Starting and Stopping Greenplum Performance Monitor ... 17
Reconfiguring Greenplum Performance Monitor ... 17
Editing the Configuration File ... 17
Reconfiguring a Monitor Instance with gpperfmon ... 18
Agent Administration ... 19
Restarting the Master Agent ... 19
Adding and Removing Monitored Hosts ... 19
Viewing and Maintaining Master Agent Log Files ... 19
Greenplum Performance Monitor Database Administration ... 20
Connecting to the Performance Monitor Database ... 20
Backing Up and Restoring the Monitor Database ... 20
Maintaining Historical Data ... 20
Web Server Administration ... 21
Starting and Stopping the Web Service ... 21
Configuring the Web Server ... 21
Viewing and Maintaining Web Server Log Files ... 21

**Chapter 4: Using Greenplum Performance Monitor** ... 23
About the Greenplum Performance Monitor console ... 23
Accessing the Greenplum Performance Monitor Console ... 23
Monitoring the Current System Workload ... 25

Viewing the System Summary ..... 25
Viewing System Metrics ..... 25
Monitoring Active Queries ..... 27
Viewing the Queries Summary ..... 27
Viewing Query Lists ..... 28
Viewing Historical System Metrics ..... 28
Viewing Historical System Metrics in the Monitor Console ..... 29
Viewing Historical System Metrics using SQL ..... 29
Viewing Historical Query Metrics ..... 30
Viewing Historical Query Metrics in the Monitor Console ..... 30
Viewing Historical Query Metrics using SQL ..... 31

**Appendix A: Utility Reference: gpperfmon** ..... 32

**Appendix B: Configuration Reference: gpperfmon.conf** ..... 34
Master Agent Options ..... 34
Greenplum Performance Monitor console Options ..... 35

**Appendix C: Greenplum Performance Monitor Catalog Reference** 37
system_now ..... 38
system_tail ..... 40
system_history ..... 42
database_now ..... 44
database_tail ..... 45
database_history ..... 46
queries_now ..... 47
queries_tail ..... 48
queries_history ..... 49
iterators_now ..... 50
iterators_tail ..... 57
iterators_history ..... 64

**Copyright © 2008 by Greenplum, Inc. All rights reserved.**

This publication pertains to Greenplum software and to any subsequent release until otherwise indicated in new editions or technical notes.

Information in this document is subject to change without notice. The software described herein is furnished under a license agreement, and it may be used or copied only in accordance with the terms of that agreement. Upgrades are provided only at regularly scheduled software release dates. No part of this publication may be reproduced, transmitted, or translated in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without the prior written permission of Greenplum, Inc. Greenplum makes no warranty of any kind with respect to the completeness or accuracy of this manual.

Greenplum, the Greenplum logo, and Greenplum Database are trademarks of Greenplum, Inc. PostgreSQL is a trademark of Marc Fournier held in trust for The PostgreSQL Development Group. All other company and product names used herein may be trademarks or registered trademarks of their respective companies.

Some Greenplum Database management tools make use of the Apache Portable Runtime library (http://www.apache.org/licenses) and the Eventlib library Copyright 2000-2004 Niels Provos and are subject to their respective license agreements.

# Preface

This guide provides information for system administrators and Greenplum Database super users responsible for administering the Greenplum Performance Monitor.


- About This Guide
- Greenplum Performance Monitor Documentation
- Contact Us


## About This Guide

This guide provides information and instructions for installing, configuring, using and maintaining Greenplum Performance Monitor. This guide assumes knowledge of Linux/Unix system administration, database management systems, database administration, and structured query language (SQL).

## Greenplum Performance Monitor Documentation

The Greenplum Performance Monitor documentation is provided in PDF. The documentation can be found in the `docs` directory of your Greenplum Performance Monitor installation, or on the Greenplum Network.

The following documents are provided with this release of Greenplum Performance Monitor:

- **Release Notes** — The release notes (`README`) explain the new features and any known issues associated with this release.
- **Greenplum Performance Monitor Guide** — This guide contains conceptual, instructional and reference information for database and system administrators responsible for installing, configuring and administering Greenplum Performance Monitor.

### Document Conventions

The following conventions are used throughout the Greenplum Database documentation to help you identify certain types of information.

### Text Conventions

**Table 0.1** Text Conventions

| Text Convention | Usage | Examples |
|---|---|---|
| **bold** | Button, menu, tab, page, and field names in GUI applications | Click **Cancel** to exit the page without saving your changes. |
| *italics* | New terms where they are defined<br>Database objects, such as schema, table, or columns names | The *master instance* is the postmaster process that accepts client connections.<br>Catalog information for Greenplum Database resides in the *pg_catalog* schema. |
| `monospace` | File names and path names<br>Programs and executables<br>Command names and syntax<br>Parameter names | Edit the `postgresql.conf` file.<br>Use `gpstart` to start Greenplum Database. |
| *`monospace italics`* | Variable information within file paths and file names<br>Variable information within command syntax | `/home/gpadmin/`*`config_file`*<br>`COPY` *`tablename`* `FROM '`*`filename`*`'` |
| **`monospace bold`** | Used to call attention to a particular part of a command, parameter, or code snippet. | Change the host name, port, and database name in the JDBC connection URL:<br>`jdbc:postgresql://`**`host:5432/mydb`** |
| `UPPERCASE` | Environment variables<br>SQL commands<br>Keyboard keys | Make sure that the Java `/bin` directory is in your `$PATH`.<br>`SELECT * FROM my_table;`<br>Press `CTRL+C` to escape. |

### Command Syntax Conventions

**Table 0.2** Command Syntax Conventions

| Text Convention | Usage | Examples |
|---|---|---|
| `{ }` | Within command syntax, curly braces group related command options. Do not type the curly braces. | `FROM { 'filename' \| STDIN }` |
| `[ ]` | Within command syntax, square brackets denote optional arguments. Do not type the brackets. | `TRUNCATE [ TABLE ] name` |
| `...` | Within command syntax, an ellipsis denotes repetition of a command, variable, or option. Do not type the ellipsis. | `DROP TABLE name [, ...]` |
| `\|` | Within command syntax, the pipe symbol denotes an "OR" relationship. Do not type the pipe symbol. | `VACUUM [ FULL \| FREEZE ]` |
| `$ system_command`<br>`# root_system_command`<br>`=> gpdb_command`<br>`=# su_gpdb_command` | Denotes a command prompt - do not type the prompt symbol. `$` and `#` denote terminal command prompts. `=>` and `=#` denote Greenplum Database interactive program command prompts (`psql` or `gpssh`, for example). | `$ createdb mydatabase`<br>`# chown gpadmin -R /datadir`<br>`=> SELECT * FROM mytable;`<br>`=# SELECT * FROM pg_database;` |

### Naming Conventions and Acronyms

**Table 0.3** Naming Conventions and Acronyms

| Convention | Meaning |
|---|---|
| `$MASTER_DATA_DIRECTORY/gpperfmon/` | The base directory for Greenplum Performance Monitor external data tables, configuration, and logs. For example:<br>`/gproot/greenplum-db/data/gp-1/gpperfmon/` |
| `$GPPERFMONHOME` | The installation directory for Greenplum Performance Monitor. This directory contains documentation, database scripts, and program files. For example:<br>`/usr/local/greenplum-perfmon-2.0.0.0` |
| `gpadmin` | The default name for the Greenplum Database super user. |
| `KB` | Kilobytes ( $2^{10}$ ) |
| `MB` | Megabytes ( $2^{20}$ ) |
| `GB` | Gigabytes ( $2^{30}$ ) |
| `TB` | Terabytes ( $2^{40}$ ) |

## Contact Us

To contact Greenplum customer support, call 1-866-410-6060 or send email to:

support@greenplum.com

# 1 About Greenplum Performance Monitor

Greenplum provides an optional performance monitoring feature that administrators can install and enable for Greenplum Database. Greenplum Performance Monitor lets administrators view the status and performance of actively running queries and allows for detailed analysis of historical queries and system metrics. A browser-based graphical console provides easy access to a variety of configurable views and reports on performance data.

## Greenplum Performance Monitor Architecture

Greenplum Performance Monitor is based on an agent architecture. Agent processes are installed on each segment host, and all other monitor components are installed on the master host.

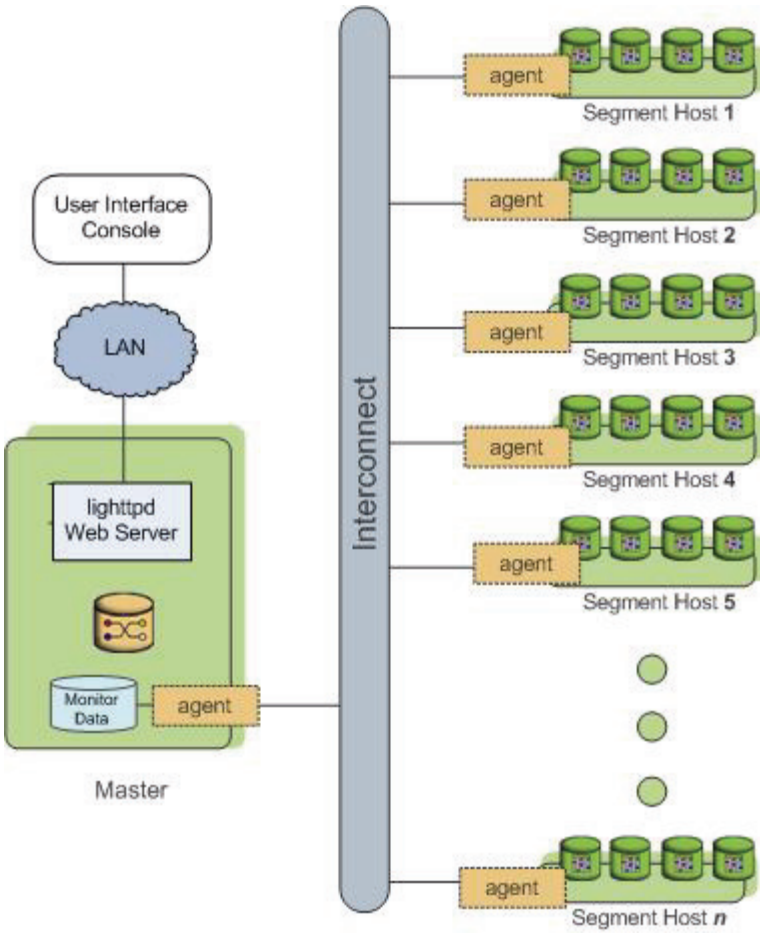



**Figure 1.1** Greenplum Performance Monitor Architecture

The agents collect performance data about active queries and system utilization and provide the data to the Greenplum master at regular intervals. The data is stored in a dedicated Greenplum Performance Monitor database on the master, where it can be accessed through the Greenplum Performance Monitor console or through SQL queries.

## Greenplum Performance Monitor Agents

Greenplum Performance Monitor agents run on all hosts in a Greenplum Database array. Each segment contains its own segment agent `gpsmon` process, which collects runtime statistics for system metrics, queries, and iterators. A master agent `gpmmon` process controls the segment agents, starting and stopping them and maintaining TCP connections.

At runtime, the master agent retrieves performance data from the segment agents at configurable intervals called `quantum`. The master agent pulls together the data from all agents, stores it in flat files, and periodically commits the data in the files to the Greenplum Performance Monitor database.

The behavior of monitor agents is affected by the `postgresql.conf` parameter, `gp_enable_gpperfmon`. When this parameter is set to `on`, the segment agents provide system, database, query and iterator data to the master agent. Without this parameter, the segment agents only provide system metrics. See Editing postgresql.conf for more information.

## Greenplum Performance Monitor Database

The Greenplum Performance Monitor database is a distributed Greenplum database dedicated to storing and serving performance data for the system. The monitor installation utility runs SQL cripts to build and initialize this database, named `gpperfmon`, on the master host.

Greenplum administrators can connect to the monitor database to manage and query performance data using client programs such as `psql` or application programming interfaces (APIs) such as JDBC and ODBC. Also, administrators can view reports on current and historical peformance data in the Greenplum Performance Monitor graphical interface.

The database consists of three sets of tables, one set for current data, a “tail” set for data in transition, and another set for historical data.

**Table 1.1** Greenplum Performance Monitor Data Tables

| Current Data Table (External) | Tail Data Table (External) | Historical Data Table |
|---|---|---|
| *system_now* | *system_tail* | *system_history* |
| *database_now* | *database_tail* | *database_history* |

**Table 1.1** Greenplum Performance Monitor Data Tables

| Current Data Table (External) | Tail Data Table (External) | Historical Data Table |
|---|---|---|
| *queries_now* | *queries_tail* | *queries_history* |
| *iterators_now* | *iterators_tail* | *iterators_history* |

### Current Data Tables

The current data is stored in flat files that are treated as external tables by the database. These files are located in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`.

For metrics, a row of data persists in the `metrics_now` table for one quantum period, after which it is replaced by the next row for that period. Query and iterator data persists in the current data tables as long as the query runs, refreshing every quantum period. After the query is finished, the data persists for one additional quantum period before passing to the tail data tables.

### Tail Data Tables

After passing out of the current data tables, all data is stored in the tail tables for a fixed period of two minutes before being committed to the historical data tables. These files are located in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`.



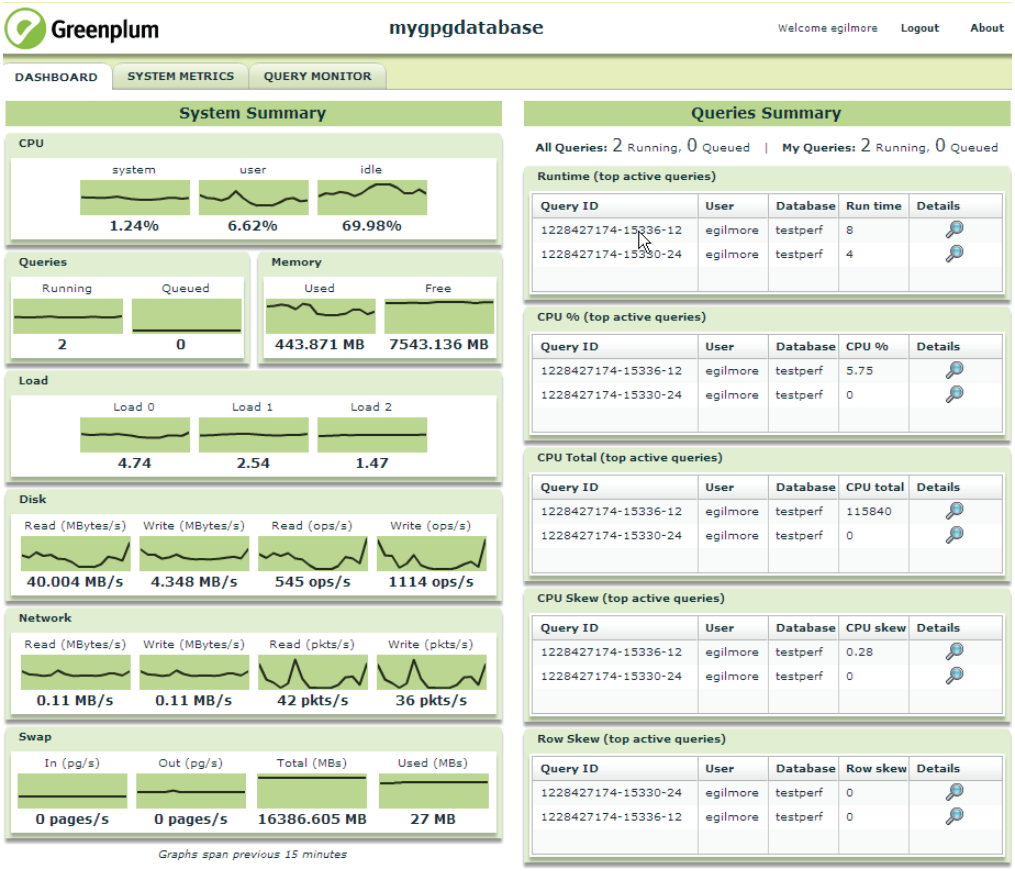
**Note:** Inside `$MASTER_DATA_DIRECTORY/gpperfmon/data/` you may see temporary files prefixed with an undercore character, such as `_queries now tail.dat`. Do not query or manipulate these temporary files, which may be safely ignored.

### Historical Data Tables

The historical data tables are distributed Greenplum Database tables in the database `gpperfmon`. These tables are partitioned by month, allowing you to drop partitions for data that you no longer need. The installation utility creates eighteen months of partitioned tables. For more information on these tables and the Greenplum Performance Monitor database schema, see Appendix C, "Greenplum Performance Monitor Catalog Reference".

## Greenplum Performance Monitor Console

Greenplum Performance Monitor provides a graphical console for viewing performance metrics. With this browser-based application, you can track the performance of active queries and view historical data for queries and system metrics.




For more information, see Chapter 4, "Using Greenplum Performance Monitor".

## Greenplum Performance Monitor Web Service

The Greenplum Performance Monitor console queries the monitor database through a web service framework composed of a lightweight Lighttpd web server and Python-based middleware. The Lighttpd server is an open-source web server with a low memory footprint and light CPU load. For more information, see http://www.lighttpd.net/.

The monitor installation utility sets up the Lighttpd web server and web service, prompting you for basic configuration information on the desired port and SSL options. Under normal conditions, the web server and web service API require minimal maintenance and administration, as described in "Web Server Administration" on page 21.

# 2. Installing Greenplum Performance Monitor

You must be the Greenplum Performance Monitor superuser to install Greenplum Performance Monitor. Installing Greenplum Performance Monitor involves the following high-level steps:

- Preparing for Installation
- Installing Greenplum Performance Monitor on the Master
- Installing the Segment Monitor Agents

This chapter also provides information about uninstalling Greenplum Performance Monitor.

## Preparing for Installation

Before installing Greenplum Performance Monitor, create a monitor superuser and edit settings in `postgresql.conf` on the master.

### Creating the Monitor Superuser

The Greenplum Performance Monitor requires a dedicated superuser role with database connection privileges. The monitor superuser owns the database that stores performance statistics data, connects to segment hosts to control agents, and performs other monitor operations.

**To create the monitor superuser and assign connection privileges:**

1. Using `psql` or another PostgreSQL-compatible client program, create a new superuser role with the `CREATEDB` attribute. For example:

   ```
   $ psql template1 -c 'CREATE ROLE gpmon WITH SUPERUSER
   CREATEDB LOGIN;'
   ```

2. Edit `$MASTER_DATA_DIRECTORY/pg_hba.conf` to give the `gpmon` superuser privileges to connect to the database. For example, to allow trusted local connections, add the following entries to the file:

   ```
   local     all     gpmon     trust
   host      all     gpmon    127.0.0.1/32    trust
   ```

3. Use `gpstop` with the `-u` option to reload `pg_hba.conf`:

   ```
   $ gpstop -u
   ```

### Editing postgresql.conf

To enable the monitor in your Greenplum environment, you must add new parameters to `postgresql.conf` in your Greenplum Database master.

**To configure Greenplum Performance Monitor:**

1. Open `$MASTER_DATA_DIRECTORY/postgresql.conf` and type in the following new parameter, set to "on" as shown:

   ```
   gp_enable_gpperfmon=on
   ```

   Optionally, you may want to organize monitor parameters under a labeled section by adding comment text similar to other sections of the file. For example:

   ```
   #------------------------------
   # GPPERFMON OPTIONS
   #------------------------------
   gp_enable_gpperfmon=on
   ```

2. If you want to use a different port than the default port 8888, add and set the following new parameter:

   ```
   gpperfmon_port=<port_number>
   ```

3. Find the following parameter in the "EXTERNAL TABLES" section of `postgresql.conf` and verify that it is set to "on" as shown:

   ```
   gp_external_enable_exec=on
   ```

4. Use `gpstop` with the `-u` option to reload `postgresql.conf`:

   ```
   $ gpstop -u
   ```



**Note:** The parameter `gp_enable_gpperfmon` is required for the collection of database, query and iterator data. The monitor collects only system metrics when this parameter is not enabled.

### System Requirements

Requirements for the Greenplum Database master and segment hosts amply accomodate the requirements of Greenplum Performance Monitor. The monitor's demands on system resources depend on system load. In typical environments, the impact of Greenplum Performance Monitor on system resources is minimal.

## Installing Greenplum Performance Monitor on the Master

Installing the monitor on the master host involves the following high-level tasks:

- Installing Greenplum Performance Monitor on the Master creates the installation directory (`$GPPERFMONHOME`) on the host machine.
- Setting up Greenplum Performance Monitor on the Master creates and configures the the agents, the database, and other monitor components on the host machine.

## About Your Greenplum Performance Monitor Installation

The installation and setup procedures create two directories on your master host: the installation directory, and a `gpperfmon` directory that is set up in your Greenplum Database master data directory. This section describes their contents and first-level subdirectories.

### Installation Directory

The following files and first-level subdirectories are copied into the installation folder that you specify in Installing Greenplum Performance Monitor on the Master:

- `gpperfmon_path.sh` — file containing environment variables for the monitor
- `gpperfmon.sql` — script used to create the monitor database
- **bin** — program files for Greenplum Performance Monitor
- **docs** — documentation, including adminstration guide and release notes
- **lib** — library files for Greenplum Performance Monitor
- **www** — web service and user interface files

After installation on the master, this directory must be copied to all segments as described in Setting up Greenplum Performance Monitor on the Master. The `gpperfmon_path.sh` variable`$GPPERFMONHOME` points to this directory on the master.

### gpperfmon Directory

The following `gpperfmon` directory contents are created in the master data directory specified in your Greenplum Database environment:

- `gpmmon.pid` -- file containing an updated process ID for the monitor
- **bin** — symbolic links to program files in the installation directory
- **conf** — monitor and web server configuration files
- **data** — .dat files and text files for external tables
- **logs** -- monitor and web server log files (if you accepted default log location)
- **web** -- symbolic links to web server files in the installation directory

## Installing Greenplum Performance Monitor on the Master

This section describes how to create the installation directory for Greenplum Performance Monitor on your master host.

#### To install Greenplum Performance Monitor

1. Download or copy the installer file to the Greenplum Database master host. Installer files are available from Greenplum for RedHat (32-bit and 64-bit) and Solaris (64-bit) platforms.
2. Unzip the installer file where *`PLATFORM`* is either `RHEL4-i386` (RedHat 32-bit), `RHEL4-x86_64` (RedHat 64-bit) or `SOL-x86_64` (Solaris 64-bit). For example:

   ```
   # unzip greenplum-perfmon-2.0.0.0-PLATFORM.zip
   ```

3. Launch the installer using `bash`. For example:

   ```
   # /bin/bash greenplum-perfmon-2.0.0.0-PLATFORM.bin
   ```

4. The installer will prompt you to accept the Greenplum Performance Monitor license agreement. Type `yes` to accept the license agreement.

5. The installer will prompt you to provide an installation path. Press `ENTER` to accept the default install path (`/usr/local/greenplum-perfmon-2.0.0.0`), or enter an absolute path to an install location. You must have write permissions to the location you specify.

6. The installation directory contains the file `gpperfmon_path.sh` with path and environment settings for the monitor. You must source this file in this directory, or the `gpadmin` user's startup shell profile (such as `.bashrc`).

   For example, you could add a line similar to the following to your chosen profile files:

   ```
   source /usr/local/greenplum-perfmon-2.0.0.0/gpperfmon_path.sh
   ```

   After editing the chosen profile file, source it as the correct user to make the changes active. For example:

   ```
   $ source ~/.bashrc
   ```

   After sourcing `gpperfmon_path.sh`, you can continue to Setting up Greenplum Performance Monitor on the Master.

## Setting up Greenplum Performance Monitor on the Master

The `gpperfmon` utility sets up Greenplum Performance Monitor on the master. This utility creates the gpperfmon Directory and sets up the following components on the master host:

- Dedicated Greenplum database for performance data (`gpperfmon`)
- Master monitor agent (`gpmmon`)
- Web server and scripts

The Greenplum Database must be running during the setup procedure for Greenplum Performance Monitor.

During the setup process, the utility prompts you for values to configure these components. For more information on choosing values for your environment, see detailed descriptions of the configuration parameters in Appendix B, "Configuration Reference: gpperfmon.conf". To accept the displayed default values for any of these parameters at configuration time, hit the `ENTER` key.

### To set up Greenplum performance monitor on the master

1. Log in as the monitor superuser.

2. With the Greenplum Database running, launch the `gpperfmon` utility with the `--setup` option. For example:

   ```
   $ gpperfmon --setup
   ```

3. The utility prompts you to create the Greenplum Performance Monitor database. Type `Y` to agree and then type `Y` again to verify. The installer runs SQL scripts to create the database and displays a message indicating that the database was created successfully.
4. Enter a value for Quantum. The default is 15 seconds.
5. Enter a value for Minimum query run time. The default is 60.
6. Enter a value for Minimum detailed query run time. The default value displayed is the value you specified for Minimum query run time.
7. Enter a location for the Greenplum Performance Monitor log files. The default is `$MASTER_DATA_DIRECTORY/gpperfmon/logs`.
8. Enter the web server port for the monitor console. The default is 28080.
9. (Optional) Enter `Y` or `N` to set up SSL connections for the monitor console. If you enter `Y`, you are prompted for the following DN information used to create an unsigned certificate for the monitor console:

```
Country Name (2 letter code) [GB]:
State or Province Name (full name) [Berkshire]:
Locality Name (eg, city) [Newbury]:, s
Organization Name (eg, company) [My Company Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (eg, your name or your server's hostname) []:
Email Address []:
```



**Note:** Because database login information is sent over the network, Greenplum recommends using SSL to encrypt these communications. For more information on SSL configuration, see "Setting up SSL Encryption" on page 25.

When the utility displays a message indicating that `gpperfmon` configuration is complete, you have successfully installed Greenplum Performance Monitor on your master host.

If you need to change the setup on your master host, you can run `gpperfmon --setup` again on the existing monitor instance. For more information, see "Reconfiguring a Monitor Instance with gpperfmon" on page 18.

For more information on next steps, see Installing the Segment Monitor Agents and Starting and Stopping Greenplum Performance Monitor.

# Installing the Segment Monitor Agents

To install segment agents, you must create a tar file of the installation directory on your master and extract it in the same directory location on all segments. This section describes how to complete these tasks in your Greenplum Database array using the utilities `gpscp` and `gpssh`. For more information on these utilities, see the *Greenplum Database Administrator Guide*.

It is important to install all segment agents in the same directory location as the installation directory on the master. For instance, if you installed to the default directory on the master, you must install all segment monitor agents to `/usr/local/greenplum-perfmon-2.0.0.0`.

Segment monitor agents do not require a setup procedure like the one performed on the master. After you install segment agents, the master agent starts and configures them automatically.

**To install Monitor Agents on the Segment Hosts**

1. On the master host, create a tar file of your Greenplum Performance Monitor installation. For example (running as `root`):

   ```
   # su -
   # cd /usr/local
   # gtar -cvf /usr/local/perfmon.tar greenplum-perfmon-2.0.0.0
   ```

2. Copy the tar file to the segment hosts using gpscp. For example:

   ```
   # gpscp -f /home/gpadmin/single_seg_hosts_file
   /usr/local/perfmon.tar =:/usr/local
   ```

3. Start an interactive session in gpssh. For example:

   ```
   # gpssh -f /home/gpadmin/single_seg_hosts_file
   ```

4. At the `gpssh` command prompt, untar the tar file in the installation directory on the segment hosts. This must be the same location as `gpperfmon` on your master host. For example:

   ```
   => gtar --directory /usr/local -xvf /usr/local/perfmon.tar
   ```

5. Confirm that the Greenplum Performance Monitor directory was installed in the correct location (the same location as `gpperfmon` on your master host). For example:

   ```
   => ls /usr/local/greenplum-perfmon-2.0.0.0
   ```

6. Change the ownership of the Greenplum Performance Monitor install directory to the `gpadmin` user or group. For example:

   ```
   => chown -R gpadmin /usr/local/greenplum-perfmon-2.0.0.0
   => chgrp -R gpadmin /usr/local/greenplum-perfmon-2.0.0.0
   ```

7. Remove the tar file. For example:

   ```
   => rm /usr/local/gp.tar
   ```

## Next Steps

Once you have installed the master agent and all segment agents, you can start the monitor as described in "Starting and Stopping Greenplum Performance Monitor" on page 17.

To immediately begin monitoring Greenplum Database metrics using the graphical console, make sure you start both the master monitor agent and the web service, and then follow the instructions in "Accessing the Greenplum Performance Monitor Console" on page 23.

If you want to view monitor data directly using SQL, follow the instructions in "Connecting to the Performance Monitor Database" on page 20. Once connected to the monitor database, you can confirm that a running monitor instance is actively collecting system metrics with a simple query like the following:

```
$ gpperfmon=# select * from system_now;
```

This query returns the current system metrics stored in the table *system_now*. Queries and views for more complex data analysis are described in Chapter 4, "Using Greenplum Performance Monitor".

## Uninstalling Greenplum Performance Monitor

To uninstall, you must remove monitor directories, drop the monitor database, and disable the monitor in your Greenplum Database configuration.

**To uninstall Greenplum Performance Monitor**

1. On the Greenplum Database master, open `postgresql.conf` and change the value of `gp_enable_gpperfmon` to `off`.
2. Reload the configuration file using the `gpstop` utility. For example:
   ```
   $ gpstop -u
   ```
3. Stop Greenplum Performance Monitor if it is currently running. For example:
   ```
   $ gpperfmon --stop
   ```
4. Remove the monitor installation directory. For example:
   ```
   $ rm -rf gpperfmon-3.2
   ```
5. Remove the directory that contains monitor data and configuration information. In a default setup this is `$MASTER_DATA_DIRECTORY/gpperfmon`. For example:
   ```
   $ rm -rf gpperfmon
   ```
6. Drop the monitor database using the `dropdb` utility. For example:
   ```
   $ dropdb gpperfmon
   DROP DATABASE gpperfmon;
   DROP DATABASE
   ```
7. Remove segment agents from the segment hosts. For example:

```
# gpssh -f /home/gpadmin/single_seg_hosts_file
=> cd /usr/local
=> rm -rf greenplum-perfmon-2.0.0.0
```

8. Remove or comment the `gpmon` user entry in `pg_hba.conf`. For example:

```
#local      all      gpmon      trust
#host       all      gpmon    127.0.0.1/32    trust
```

9. Drop the monitor user role from the database. For example:

```
template1=# DROP ROLE gpmon
```

# 3 Administering Greenplum Performance Monitor

Routine administration of Greenplum Performance Monitor is performed using the `gpperfmon` utility. This utility enables basic tasks such as stopping and restarting the monitor. For more information about `gpperfmon`, see "Utility Reference: gpperfmon" on page 32, or view usage information with the command `gpperfmon -?`.

Other tasks for administration of the web server or the monitor database are performed with native tools or Greenplum utilities, as described in this chapter.

## Starting and Stopping Greenplum Performance Monitor

The `gpperfmon` utility starts, stops and restarts Greenplum Performance Monitor with the following option flags:

- `--start`
- `--stop`
- `--restart`

For example, to start Greenplum Performance Monitor, run the following command:

```
$gpperfmon --start
```

These command options control the master agent, the segment agent processes, and the UI web server. If you want to exclude the UI web server from these commands, add the flag `--noweb`. For example, to restart agents without restarting the web server:

```
$gpperfmon --restart --noweb
```

## Reconfiguring Greenplum Performance Monitor

Configuration parameters for Greenplum Performance Monitor are stored in `$MASTER_DATA_DIRECTORY/gpperfmon/conf/gpperfmon.conf`. This configuration file contains all the configuration options described in "Configuration Reference: gpperfmon.conf" on page 34.

If you want to change configuration values for a monitor instance, edit `gpperfmon.conf`, or run `gpperfmon --setup`.

### Editing the Configuration File

You can directly edit `gpperfmon.conf` to adjust a single value such as `min_detailed_query_time`.

#### To edit the configuration file

1. In a text editor, open `$MASTER_DATA_DIRECTORY/gpperfmon/conf/gpperfmon.conf`.

2. Locate the desired parameter and type the new value. For example, to change `min_detailed_query_time`, modify this setting found in the `[GPMMON]` section of the file:

   ```
   min_detailed_query_time=360
   ```

3. When you are finished making changes, save `gpperfmon.conf`.

4. Restart the monitor. For example:

   ```
   $ gpperfmon --restart
   ```

## Reconfiguring a Monitor Instance with gpperfmon

Running `gpperfmon --setup` provides a convenient way to change one or all of the configuration values for a monitor instance. For each parameter prompt, the utility displays the existing value as the default.

Also, with `gpperfmon --setup` you can replace the existing monitor database with a fresh instance, or change values that are not available in `gpperfmon.conf`, such as the web server port and the DN values used for SSL.

### To reconfigure a monitor instance with gpperfmon

1. Run `gpperfmon --setup`.

2. The utility displays the message warning that a monitor configuration file already exists. Type `Y` to reconfigure the monitor instance.

3. The utility prompts you to create a new database, warning you that all existing performance data will be lost. Type `Y` or `N` as desired.

4. Enter a value for Quantum.

5. Enter a value for Minimum query run time.

6. Enter a value for Minimum detailed query run time.

7. Enter a location for the Greenplum Performance Monitor log files.

8. Enter the web server port for the monitor console.

9. (Optional) Enter `Y` or `N` to set up SSL connections for the monitor console. If you enter `Y`, you are prompted for the following DN information:

   ```
   Country Name (2 letter code) [GB]:
   State or Province Name (full name) [Berkshire]:
   Locality Name (eg, city) [Newbury]:, s
   Organization Name (eg, company) [My Company Ltd]:
   Organizational Unit Name (eg, section) []:
   Common Name (eg, your name or your server's hostname) []:
   Email Address []:
   ```



**Note:** Because database login information is sent over the network, Greenplum recommends setting up SSL to encrypt these sensitive communications. For more information on SSL configuration, see "Setting up SSL Encryption" on page 25.

When the utility displays a message indicating that `gpperfmon` configuration is complete, you have successfully reconfigured Greenplum Performance Monitor.

# Agent Administration

This section describes basic agent administration tasks, including setting up a cron job to restart the master agent in case of failure, adding monitor hosts, and viewing agent log files.

## Restarting the Master Agent

In this release, the master agent is not automatically restarted if it fails. Greenplum recommends setting up a cronograph job on the master host to start the master agent every few minutes. If the agent is already running, the cron job fails. If the agent is not running, the cron job restarts it.

## Adding and Removing Monitored Hosts

Segment agents on new hosts are detected automatically by the master agent. To add a new monitored host, follow the procedure described in "Installing the Segment Monitor Agents" on page 14 . The master agent automatically detects, starts, and begins harvesting data from the segment agent.

To verify the addition of a new monitored host, you can check for the new hostname in the Greenplum Performance Monitor console **System Metrics** view described in "Viewing Realtime Metrics by Server" on page 26. Alternately, you can query the *system_now* table for the row containing current metrics for the host. For example:

```
gpperfmon=# select * from system_now where hostname =
'<new_hostname>';
```

To remove a segment agent, remove the installation directory. For example:

```
rm -rf greenplum-perfmon-2.0.0.0
```

## Viewing and Maintaining Master Agent Log Files

Log messages for the master agent are written to `$MASTER_DATA_DIRECTORY/gpperfmon/logs/gpmmon.log` by default. To change the log file location, run `gpperfmon -- setup`, or edit the `log_location` parameter in `gpperfmon.conf`. For more information, see "Reconfiguring Greenplum Performance Monitor" on page 17.

If the parameter `verbose` is set to the default value of `0`, only fatal errors are logged. To log information for troubleshooting or debugging, set this value to 1 or 2 for detailed messages.

At higher logging levels, the size of the log files may grow dramatically. If the log files grow to an unmanageable size, you can restart the monitor to clear out existing monitor log messages.

# Greenplum Performance Monitor Database Administration

As a distributed Greenplum database, the monitor database interoperates with the set of administration tools provided with your Greenplum installation. This section describes how to use Greenplum utilities in the routine maintenance of the monitor database.

With `gpperfmon --setup` you can replace the existing monitor database with a fresh instance. However, this process results in the loss of all existing performance data in the `gpperfmon` database.

## Connecting to the Performance Monitor Database

Connect to the `gpperfmon` database as you would connect to any Greenplum Database, using a Postgresql-compatible client. Administrators using `psql` can connect to the monitor database on the usual Greenplum Database port on the master host. For example:

```
$ psql -d gpperfmon -h master_host -p 5432 -U gpadmin
```

## Backing Up and Restoring the Monitor Database

You can back up and restore using the `gpdump` and `gprestore` utilities as described in chapter 21 of the *Greenplum Database Adminstrator Guide*, “Backing Up and Restoring Databases.” This document can be found in the `<installation_directory>/docs` directory of your Greenplum Database installation, or on the Greenplum Network.

Because the monitor database has a low number of tables, you may prefer to devise a backup plan using the table-level backup features of `gpdump`. For example, you can create scripts to run `gpdump` to back up the monthly partitions of the historical data tables on a monthly schedule. Alternately, you can back up your monitor database at the database level.

## Maintaining Historical Data

The historical data tables are partitioned by month, allowing you to drop partitions for data that you no longer need. The `gpperfmon` installer creates eighteen months of partitioned tables. When that period is over, you may want to create new monthly partitions. For more information on partitioning Greenplum Database tables, see chapter 14 of the *Greenplum Database Adminstrator Guide*, “Defining Database Objects.”

# Web Server Administration

The Lighttpd web server and web service middleware are installed in the `www` directory of your Greenplum Performance Monitor installation. For detailed information on Lighttpd administration, see http://www.lighttpd.net/.

## Starting and Stopping the Web Service

The `gpperfmon` utility starts, stops and restarts the Lighttpd web server together with the monitor agents, using the following option flags:

```
--start
--stop
--restart
```

For example, to start the monitor and the web service:

```
gpperfmon --start
```

If you want to stop the monitor and run only the web service, use the `--noweb` option. For example:

```
gpperfmon --stop --noweb
```

## Configuring the Web Server

The Lighttpd web server's configuration is stored in `$MASTER_DATA_DIRECTORY/gpperfmon/conf/lighttpd.conf`. Some of the parameters in this configuration file are set by the installation utility, including the web server port and SSL options described in this section.

**`server.port`**

Specifies the port for the web server that provides the monitor console and web API. Default is 28080.

**`ssl.engine`**

Determines whether SSL encryption is used for client connections. Valid values are `enable` and `disable`. If you enable SSL at installation time, this is set to `enable`.

**`ssl.pemfile`**

Specifies the path to the PEM file for SSL support. If you enable SSL at installation time, this parameter points to a self-signed certificate. If you use a trusted signed certficate, you must specify it with this parameter.

For more information on SSL configuration, see Lighttpd documentation.

## Viewing and Maintaining Web Server Log Files

Logs of web server access and server errors are written to `$MASTER_DATA_DIRECTORY/gpperfmon/logs/`. These two logs are:

- lighttpd-access.log

- lighttpd-error.log

If you experience errors viewing the Greenplum Performance Monitor console, refer to these logs for more information.

# 4 Using Greenplum Performance Monitor

Greenplum Performance Monitor provides immediate visibility into key areas affecting database performance. You can view system metrics such as CPU usage and network utilization. You can check the status of currently running queries, drilling down to iterator-level detail as desired. You can analyze historical data to identify trends or anomalies in the performance over time of the Greenplum Database system.

The browser-based console for Greenplum Performance Monitor provides an easy way to access performance data. This chapter gives information on getting started with the monitor console, as well as some guidance in the use of direct SQL queries and views on performance data.

## About the Greenplum Performance Monitor console

The Greenplum Performance Monitor console is supported for the following browsers with Macromedia Flash:

- Internet Explorer for Windows XP and Vista
- Mozilla Firefox for Windows and Linux
- Apple Safari for Macintosh

The monitor console runs on a lighttpd web server installed on the master host. The default port id `28080`. For more information about the web server, see "Web Server Administration" on page 21. The web service API for the monitor console is installed in `$GPPERFMONHOME/www`.

### Accessing the Greenplum Performance Monitor Console

You must be a Greenplum Database administrator or superuser to use the Greenplum Performance Monitor console. Superusers can view information for all queries in the system, while administrators may monitor only their own queries.

To access the monitor console, open a supported browser and navigate to the login page. For example, if you are using the default web server port of 28080 to connect with SSL, use this address:

```
https://<master_host>:28080/
```

On the login page, enter your credentials and click **Login**. This opens the **Dashboard** page of the Greenplum Performance Monitor console.

The **Dashboard** page of the console provides a graphical system snapshot and a summary view of active queries. This page is shown in "Greenplum Performance Monitor Console" on page 7. For more information about these views, see Viewing the System Summary and Viewing the Queries Summary.

### Setting up Password Authentication

By default, administrators logging on to the console are required to provide an md5-encrypted Greenplum Database password. If the administrators using the Greenplum Performance Monitor console do not already have md5-encrypted passwords for logging on to the database, you must assign passwords using CREATE ROLE or ALTER ROLE. For example:

```
$ template1=# ALTER ROLE gpadmin WITH ENCRYPTED PASSWORD
'password1234';
```

For more information, see "Access Control and Security" in the *Greenplum Database Administrator Guide*.

You can bypass password authentication by using trust login. However, Greenplum recommends password authentication with SSL for connections to the Greenplum Performance Monitor console.

To enable password authentication for the console, you must make sure that `pg_hba.conf` enforces md5-encrypted passwords for database login, and you must set the variable `PGHOST_LOGON` to ensure that locally trusted logins on the master host are not used.

#### To set up password authentication for the console

1. Open the file `$MASTER_DATA_DIRECTORY/pg_hba.conf` in a text editor.
2. Make sure that the list includes an entry enforcing md5-encrypted passwords for database login. For example:

   ```
   host    all     gpadmin     0.0.0.0/0     md5
   ```

   For more information, see "Configuring Client Authentication" in the *Greenplum Database Administrator Guide.*
3. Save and close the file.
4. Reload the configuration files for your changes to take effect:

   ```
   $ gpstop -u
   ```
5. Open the profile file for the account that runs the `gpperfmon` utility, and add the `PGHOST_LOGON` variable set to the IP address of the master host. For example, where the master host IP address is `10.9.8.101`, add the following:

   ```
   export PGHOST_LOGON = 10.9.8.101
   ```



**Note:** This configuration requires that you do not allow trusted access to the IP address for the master host. You may still allow trusted local access to 127.0.0.1, but not to the explicit IP address.

6. Save the profile file and source it as the correct user to make the changes active. For example:

   ```
   $ source ~/.bashrc
   ```

   or

   ```
   $ source /etc/profile
   ```

**Trust Login**. To allow access to any trusted username, set `allow_trust_logon` to `yes` in the Greenplum Performance Monitor configuration file. This configuration allows any trusted username to log in to the monitor console. This unrestrictive authentication method is recommended only for testing purposes, and should not be used with sensitive data in production environments.

### Setting up SSL Encryption

If you elect to enable SSL at installation time, the installer creates a self-signed certificate for the Greenplum Performance Monitor console and uses Open SSL encryption for all connections to the server.

Because this is a self-signed certificate, supported browsers may need to be configured to accept or allow it as a security exception. This does not detract from the added security gained from encrypting communications over the connection between the console and the web server.

Optionally, you can obtain and use a certificate signed by a trusted certificate authority such as Verisign. If you use a trusted certificate, set the `lighttpd.conf` parameter `ssl.pemfile` to point to the location of the certificate. For example:

```
ssl.pemfile = "/usr/local/gpdata/gp-1/gpperfmon/conf/cert1.pem"
```

For more information on the lighttpd configuration file, see "Web Server Administration" on page 21.

## Monitoring the Current System Workload

Greenplum Performance Monitor collects current system data at every `quantum` interval. These metrics include CPU load averages, memory and swap space usage, and network and disk usage. For a full listing of the metrics collected, see Appendix C, "Greenplum Performance Monitor Catalog Reference".

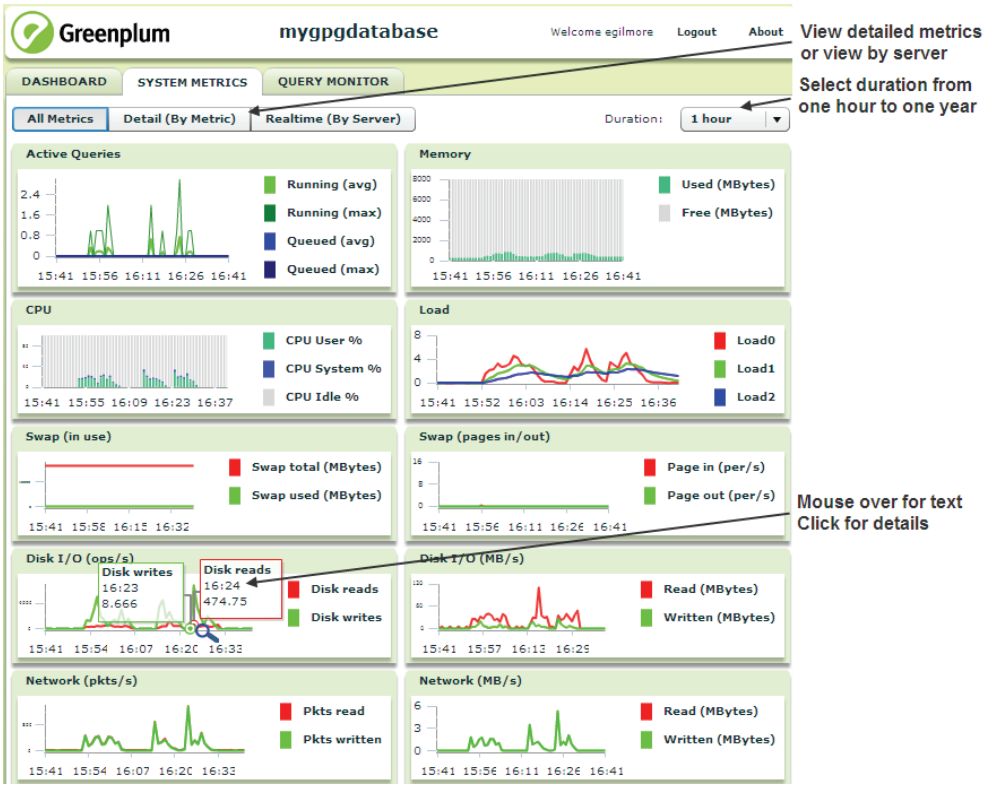
The Greenplum Performance Monitor console provides several ways to view the current system workload in the **Dashboard** and **System Metrics** pages.

### Viewing the System Summary

The **Dashboard** page of the monitor console provides a quick view of the current system workload. The left pane of this page is a **System Summary** comprised of line graphs for key system metrics such as CPU and swap page usage over the previous fifteen-minute period. Mousing over points in these line graphics displays a small window with numeric metric values per minute.

### Viewing System Metrics

For a more detailed graphical view of selected system metrics, select the **System Metrics** tab. This page includes detailed line graphs that highlight active queries, memory usage, CPU usage, swap page usage, disk I/O and network throughput.




### Viewing All Metrics

The default view for the **System Metrics** page is **All Metrics**, which charts data aggregated across all segments in the database array for a one-hour time period. If you want to increase the time period shown in the graphs, use the controls at the top left of the page. Mousing over points in these line graphics displays a small window with the exact numeric metric value, and clicking on any point opens the detail pages described in Viewing Detailed Information by Metric.

### Viewing Realtime Metrics by Server

The **Realtime (By Server)** view displays metrics by server, with a separate bar graph for each server in the array. Mousing over the end point of these bar graphs displays a small window with the exact numeric value for the metric.

### Viewing Detailed Information by Metric

To view detailed information for a given metric, you can either click on a point in the summary graphs, or select the **Detail (by Metric)** view from the upper left of the **System Metrics** page. For each metric type, there is a detail page that highlights a line graph in the upper pane and listed details in the bottom pane.

For example, the detail page for disk I/O over the past six hours displays a line graph with MB written tracked by a green line, and MB read tracked by a red line. Mousing over the lines displays data for that exact interval, and clicking on that point displays the text data in list format in the bottom pane.

To view detailed information for different metrics, select from the dropdown list at the top left of the **Detail (By Metric)** page. You can select from the following metric types:

- Active Queries
- Memory
- CPU
- Load
- Swap (in use)
- Swap (pages in/out)
- Disk (packets)
- Disk (MB)
- Network (packets)
- Network (MB)

# Monitoring Active Queries

Greenplum Performance Monitor collects data on currently running queries at every `quantum` interval, updating the metrics at each interval until the query is completed. Current query metrics include the username of the user running the query, the skew of CPU usage across segments, and detailed information for individual iterators executing the query plan.

Greenplum Performance Monitor also collects summary data for the total number of queries, active queries, and queued queries. For a full listing of the metrics collected for queries and iterators, see Appendix C, "Greenplum Performance Monitor Catalog Reference".

The Greenplum Performance Monitor console provides several ways to view active query data in the **Dashboard** and **Query Monitor** pages.

## Viewing the Queries Summary

The monitor console **Dashboard** page provides a summary view of active queries for the database. The **Queries Summary** displays the following lists of top currently running queries sorted by key metrics:

- **Runtime** Current queries sorted by runtime.
- **CPU %** Current queries sorted by percentage of CPU consumed.
- **CPU Total** Current queries sorted by the sum of CPU usage taken from all segments. For more information, see *cpu_elapsed*.
- **CPU Skew** Current queries sorted by CPU skew, which reflects the variation in CPU used in running the query across all segments in the database array. For more information, see *skew_cpu*.

- **Row Skew** Current queries sorted by row skew, which reflects the variation in rows in for the query across all segments in the database array. For more information, see *skew_rows*.

To view more detail on any query listed in the **Queries Summary**, click on the list item. This opens a view of the query details, with options to drill down to the query plan.

### Viewing Query Lists

For a detailed list view of currently running queries, select the **Query Monitor** tab. The default view for this page is **Active — all queries**, which displays a list view of all active queries. You can select alternate views listing only queries submitted by your own admin user account (**Active — my queries**) or queries that you have selected to watch (**Watched queries**).

Each list item in these views includes username, database, status, wait time, and other useful query data fields. Clicking on any list item displays the query text in the bottom pane of this page. To watch a query, check the box in the **Watch** column at the left of the query list item.

You can search for queries to retrieve a list filtered by your search criteria. See "Searching for Queries" on page 30.

#### Viewing Query Details

To view greater detail for a listed query, double-click the list item or click the icon in the **Details** column. This opens a page with query details, plus an option to display a graphical view of the query plan. In the **Query Plan** view, you can click on a node type in the left pane to display full detail for the iterator.



**Note:** If you can't view detailed information for iterators in a query plan, the query may not have run beyond the threshold determined by `min_detailed_query_time`. Check the query runtime against the value of this parameter.

## Viewing Historical System Metrics

Historical tables in the monitor database store all system data collected by the monitor. You can analyze this data to identify trends in the performance of the Greenplum Database system over time.

With a default `quantum` setting of 15 seconds, the monitor collects very large amounts of historical system data. To view meaningful groupings of system data records, you can view the data by selected time intervals with the monitor console, or with a SQL view like the one described in this section.

## Viewing Historical System Metrics in the Monitor Console

The **All Metrics** view in the **System Metrics** page displays graphs in intervals up to a year in length. If you want to increase the time period shown in the graphs, use the controls at the top left of the page. For more information, see "Viewing System Metrics" on page 25.

To view system metrics for time periods beyond a year, query the monitor database with SQL as described in the next section.

## Viewing Historical System Metrics using SQL

This section provides an example SQL view for viewing aggregated system metrics. In this example view named *system_metrics_aggregate_1min*, rows of system metrics from *system_history* are averaged over one-minute intervals and aggregated across all segments in the array:

```
drop view if exists system_metrics_aggregate_1min;

create view system_metrics_aggregate_1min as (
    select date_trunc('minute', ctime) as sample_time,
    hostname,
    avg(mem_total) as mem_total, avg(mem_used) as mem_used,
    avg(mem_actual_used) as mem_actual_used,
    avg(mem_actual_free) as mem_actual_free,
    avg(swap_total) as swap_total, avg(swap_used) as swap_used,
    avg(swap_page_in) as swap_page_in,
    avg(swap_page_out) as swap_page_out,
    avg(cpu_user) as cpu_user, avg(cpu_sys) as cpu_sys,
    avg(cpu_idle) as cpu_idle,
    avg(load0) as load0, avg(load1) as load1,
    avg(load2) as load2,
    avg(disk_ro_rate) as disk_ro_rate,
    avg(disk_wo_rate) as disk_wo_rate,
    avg(disk_rb_rate) as disk_rb_rate,
    avg(disk_wb_rate) as disk_wb_rate,
    avg(net_rp_rate) as net_rp_rate,
    avg(net_wp_rate) as net_wp_rate,
    avg(net_rb_rate) as net_rb_rate,
    avg(net_wb_rate) as net_wb_rate

    from system_history

    group by hostname, sample_time

);
```

To change the interval for averaging results, adjust the the value specified for `date_trunc`. Valid values are `microseconds`, `milliseconds`, `second`, `minute`, `hour`, `day`, `week`, `month`, `quarter`, `year`, `decade`, `century` and `millennium`.

# Viewing Historical Query Metrics

Historical tables in the monitor database store all query and iterator data collected by the monitor. You can analyze this data to identify trends in the performance of Greenplum Database over time.

To view historical query metrics, use the console search tool to retrieve lists filtered by time period, or create a SQL view as described in this section.

## Viewing Historical Query Metrics in the Monitor Console

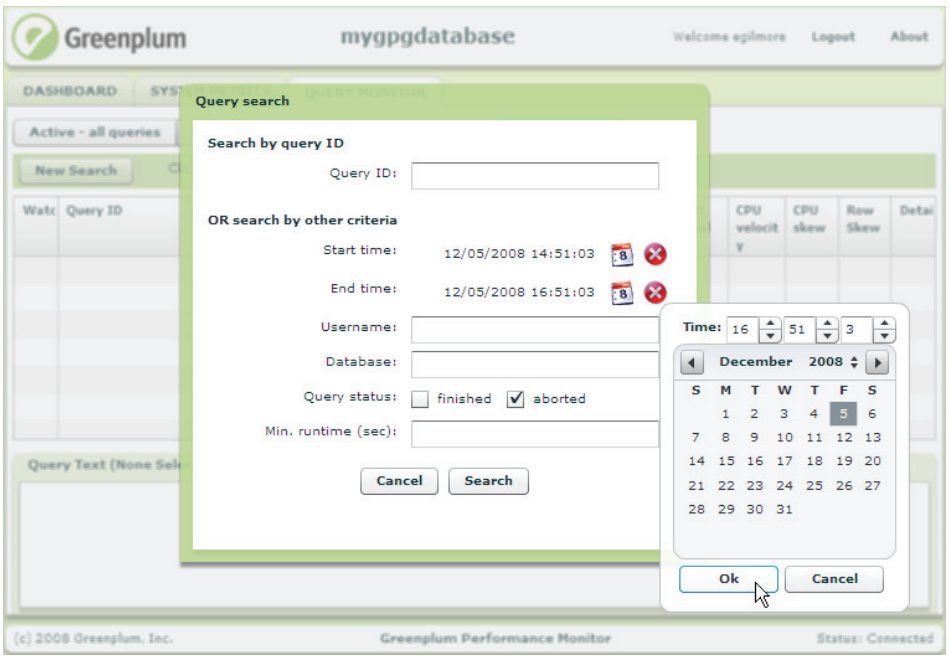
The console search tool can retrieve lists of historical query metrics filtered by specified time intervals.

### Searching for Queries

You can search completed queries and retrieve a list of queries matching your search criteria. The search results list has the same information and drill-down capablity as active query lists. For more information, see "Viewing Query Details" on page 28.

**To search completed queries:**

1. From any view in the **Query Monitor** page, click **Query search**. An empty search results page opens, with a **New Search** button located at the upper left.
2. Click **New Search**. This opens the query search screen:
3. Enter a query ID, or, alternately, enter search criteria. If you enter a query ID, the search utility ignores any other criteria specified. If you search by other criteria, you must enter at least a start time.



4. Click **Search**. The results are displayed in a results list with a maximum of 250 results per search. If the result list is too large to be useful, try narrowing your search criteria.

## Viewing Historical Query Metrics using SQL

Calculating the average, minimum and maximum values for system-wide query statistics aggregated over time can yield a useful view of database utilization. This section provides an example SQL view for viewing query statistics in this way.

In this example view named *database_metrics_1hour*, rows of statistics from the table *database_history* are averaged over hour-long intervals:

```
create view database_metrics_1hour as (

    select date_trunc('hour', ctime) as sample_time,
    avg(queries_total) as queries_total,
    min(queries_total) as queries_total_min,
    max(queries_total) as queries_total_max,
    avg(queries_running) as queries_running,
    min(queries_running) as queries_running_min,
    max(queries_running) as queries_running_max,
    avg(queries_queued) as queries_queued,
    min(queries_queued) as queries_queued_min,
    max(queries_queued) as queries_queued_max

    from database_history

    group by sample_time

);
```

To change the interval for averaging results, adjust the the value specified for `date_trunc`. Valid values are `microseconds`, `milliseconds`, `second`, `minute`, `hour`, `day`, `week`, `month`, `quarter`, `year`, `decade`, `century` and `millennium`..

# A. Utility Reference: gpperfmon

This utility controls various aspects of the Greenplum Performance Monitor, including configuration and starting and stopping monitor agents.

## Synopsis

```
gpperfmon [--setup] [--start] [--stop] [--restart] [--status]
[--version] [--noweb]
```

## Description

The `gpperfmon` utility controls various aspects of Greenplum Performance Monitor. This utility configures the monitor components at installation time, starts and stops the monitor, and provides status and version information.

For starting and stopping the monitor, `gpperfmon` controls the master monitor agent, which passes commands to all segment agents. Commands for starting and stopping also control the web service, unless the --noweb options is specified.

## Options

**`--setup`**

Configures monitor components on the master host. With this option, `gpperfmon` prompts for values to configure monitor components and writes the values to `gpperfmon.conf` and `lighttpd.conf`. For more information on these configuration parameters, see Appendix B, "Configuration Reference: gpperfmon.conf".

**`--start`**

Starts the web service and master agent, which in turn starts all segment monitor agents in the array. Can be used with the flag `--noweb` to start only the monitor agents without starting the web service.

**`--stop`**

Stops the web service and master agent, which in turn stops all segment monitor agents in the array. Can be used with the flag `--noweb` to stop only the monitor agents without stopping the web service.

**`--restart`**

Restarts the web service and master agent, which in turn restarts all segment monitor agents in the array. Can be used with the flag `--noweb` to restart only the monitor agents without restarting the web service.

This flag is used to reload configuration files after editing `gpperfmon.conf` or `lighttpd.conf`.

**--status**

Displays the status, either `Running` or `Stopped`, of the monitor and web service.

**--version**

Displays the version of the web service, master agent, and segment monitor agents.

**--noweb**

Used with `--start`, `--stop` and `--restart` to perform the command for the monitor agents only, excluding the web service.

## Examples

Start the Greenplum Monitor master agent but not the web service:

```
gppperfmon --start --noweb
```

Start the utility in order to install or to change configuration values:

```
gpperfmon --setup
```

Check the status of the monitor and web service:

```
gpperfmon --status
```

## See Also

Greenplum Performance Monitor Catalog Reference

# B. Configuration Reference: gpperfmon.conf

Settings that determine the behavior of Greenplum Performance Monitor are stored in `$MASTER_DATA_DIRECTORY/gpperfmon/conf/gpperfmon.conf`. Any Greenplum Database user with write permissions to this directory can edit `gpperfmon.conf`. For configuration changes to take effect, you must save `gpperfmon.conf` and then restart the monitor with `gpperfmon --restart`.

## Master Agent Options

These options for the master agent process `gpmmon` are set by `gpperfmon --setup`.

### quantum

Specifies the time in seconds between updates from performance monitor agents on all segments. Defaults to 15 seconds, with valid values limited to the range between 10 and 60 seconds.

If you prefer a less granular view of performance, or want to collect and analyze minimal amounts of data for system metrics, choose a higher quantum. To collect data more frequently, choose a lower value.

### min_query_time

Specifies the minimum query run time in seconds for statistics collection. The monitor logs all queries that run longer than this value in the `queries_history` table. For queries with shorter run times, no historical data is collected. Defaults to 60 seconds.

If you know that you want to collect data for all queries, you can set this parameter to a low value. Setting the minimum query run time to zero, however, collects data even for the numerous queries run by the monitor itself, creating a large amount of data that may not be useful.

### min_detailed_query_time

Specifies the minimum iterator run time in seconds for statistics collection. The monitor logs all iterators that run longer than this value in the `iterators_history` table. For iterators with shorter run times, no data is collected. Minimum value is 10 seconds.

Defaults to the value of `min_query_time`, and must always be equal to, or greater than, the value of `min_query_time`. Setting `min_detailed_query_time` higher than `min_query_time` allows you to log detailed iterator data only for especially complex, long-running queries, while still logging basic query data for shorter queries.

Given the complexity and size of iterator data, you may want to adjust this parameter according to the size of data collected. If the *iterators_history* tables are growing to excessive size without providing useful information, you can raise the value of this parameter to log iterator detail for fewer queries.

### log_location

Specifies a directory location for monitor log files. Default is `$MASTER_DATA_DIRECTORY/gpperfmon/logs`.

### verbose

Verbose mode shows progress and status messages of the performance tests as they are run. The default is `0`, which displays and logs no progress or status messages. Valid values of `1` and `2` display increasingly detailed messages, and are recommended only for troubleshooting purposes. These higher settings cause dramatic growth in log file size.

## Greenplum Performance Monitor console Options

These options for the monitor console and web server are stored in `gpperfmon.conf`. The related parameter `server.port` is stored in `lighttpd.conf`. For more information about this parameter, see "Web Server Administration" on page 21.

### server name

Specifies the server name displayed on the login page for the Greenplum Performance Monitor console. This value can be any name you want to display to users, expressed as a text string. Defaults to the actual hostname of the server.

### allow_trust_logon

Determines whether to allow access to the Greenplum Performance Monitor console for any user listed in `pg_hba.conf` with an authentication type of "trust." When set to "yes," all trusted usernames are allowed to access the monitor without a password. By default, this parameter is commented out in `gpperfmon.conf`.

### remote

Determines whether the monitor console accesses current data files (such as *system_now* or *queries_now*) directly on the file system, or through the Greenplum Database. Direct access to the file system has a significant performance advantage, but some firewall and security configurations may require database connections.

The default value of `no | False` allows direct access to the file system. Changing this parameter to `yes | True` forces the monitor console to connect to the database to query current data tables.

# C. Greenplum Performance Monitor Catalog Reference

This is a reference of the system catalog tables for Greenplum Performance Monitor.

The monitor database stores each performance statistic type (system metrics, database information, queries, and iterators) in three separate sets of tables: one for current statistics, one for "tail" data in transition, and one for historical data.


- system_now
- system_tail
- system_history
- database_now
- database_tail
- database_history
- queries_now
- queries_tail
- queries_history
- iterators_now
- iterators_tail
- iterators_history

## system_now

The table *system_now* contains current system utilization metrics. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/` . Current system utilization data is stored in *system_now* during the period between data collection from the monitor agents and automatic commitment to the *system_history* table.

Values for memory, I/O and swap space metrics are expressed in kilobytes.

**Table C.1** gpperfmon.system_now

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| hostname | varchar(64) | Hostname of the system to which the metric belongs. |
| mem_total | bigint | Total system memory for the system to which the metric belongs, including actual memory and swap space. |
| mem_used | bigint | Used system memory for the system to which the metric belongs. |
| mem_actual_used | bigint | Used actual memory (memory not including portion reserved for cache and buffers). |
| mem_actual_free | bigint | Free actual memory (memory not including portion reserved for cache and buffers). |
| swap_total | bigint | Total swap space for the system to which the metric belongs. |
| swap_used | bigint | Used swap space for the system to which the metric belongs. |
| swap_page_in | bigint | Number of swap pages in. |
| swap_page_out | bigint | Number of swap pages out. |
| cpu_user | float | CPU usage by the user. |
| cpu_sys | float | CPU usage for the system. |
| cpu_idle | float | Idle CPU capacity at metric collection time. |
| load0 | float | CPU load average for the prior one-minute period. |
| load1 | float | CPU load average for the prior five-minute period. |
| load2 | float | CPU load average for the prior fifteen-minute period. |
| quantum | int | Interval between metric collection for this metric entry. |
| disk_ro_rate | bigint | Disk read operations per second. |
| disk_wo_rate | bigint | Disk write operations per second. |
| disk_rb_rate | bigint | Bytes per second for disk read operations. |
| disk_wb_rate | bigint | Bytes per second for disk write operations. |
| net_rp_rate | bigint | Packets per second on the system network for read operations. |
| net_wp_rate | bigint | Packets per second on the system network for write operations. |

**Table C.1** gpperfmon.system_now

| column | type | description |
|---|---|---|
| net_rb_rate | bigint | Bytes per second on the system network for read operations. |
| net_wb_rate | bigint | Bytes per second on the system network for write operations. |

## system_tail

The table *system_tail* contains system utilization metrics queued for commitment to *system_history.* This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. System utilization data is stored in *system_tail* between flushing from *system_now* and automatic commitment to *system_history*.

Values for memory, I/O and swap space metrics are expressed in kilobytes

**Table C.1** gpperfmon.system_now

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| hostname | varchar(64) | Hostname of the system to which the metric belongs. |
| mem_total | bigint | Total system memory for the system to which the metric belongs, including actual memory and swap space. |
| mem_used | bigint | Used system memory for the system to which the metric belongs. |
| mem_actual_used | bigint | Used actual memory (memory not including portion reserved for cache and buffers). |
| mem_actual_free | bigint | Free actual memory (memory not including portion reserved for cache and buffers). |
| swap_total | bigint | Total swap space for the system to which the metric belongs. |
| swap_used | bigint | Used swap space for the system to which the metric belongs. |
| swap_page_in | bigint | Number of swap pages in. |
| swap_page_out | bigint | Number of swap pages out. |
| cpu_user | float | CPU usage by the user. |
| cpu_sys | float | CPU usage for the system. |
| cpu_idle | float | Idle CPU capacity at metric collection time. |
| load0 | float | CPU load average for the prior one-minute period. |
| load1 | float | CPU load average for the prior five-minute period. |
| load2 | float | CPU load average for the prior fifteen-minute period. |
| quantum | int | Interval between metric collection for this metric entry. |
| disk_ro_rate | bigint | Disk read operations per second. |
| disk_wo_rate | bigint | Disk write operations per second. |
| disk_rb_rate | bigint | Bytes per second for disk read operations. |
| disk_wb_rate | bigint | Bytes per second for disk write operations. |
| net_rp_rate | bigint | Packets per second on the system network for read operations. |
| net_wp_rate | bigint | Packets per second on the system network for write operations. |

**Table C.1** gpperfmon.system_now

| column | type | description |
|---|---|---|
| net_rb_rate | bigint | Bytes per second on the system network for read operations. |
| net_wb_rate | bigint | Bytes per second on the system network for write operations. |

## system_history

The table *system_history* stores historical system utilization metrics. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need.

Values for memory, I/O and swap space metrics are expressed in kilobytes

**Table C.1** gpperfmon.system_history

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| hostname | varchar (64) | Hostname of the system to which the metric belongs. |
| mem_total | bigint | Total system memory for the system to which the metric belongs. |
| mem_used | bigint | Used system memory for the system to which the metric belongs. |
| mem_actual_used | bigint | Used actual memory (memory not including portion reserved for cache and buffers). |
| mem_actual_free | bigint | Free actual memory (memory not including portion reserved for cache and buffers). |
| swap_total | bigint | Total swap space for the system to which the metric belongs. |
| swap_used | bigint | Used swap space for the system to which the metric belongs. |
| swap_page_in | bigint | Number of swap pages in. |
| swap_page_out | bigint | Number of swap pages out. |
| cpu_user | float | CPU usage by the user. |
| cpu_sys | float | CPU usage for the system. |
| cpu_idle | float | Idle CPU capacity at metric collection time. |
| load0 | float | CPU load average for the prior one-minute period. |
| load1 | float | CPU load average for the prior five-minute period. |
| load2 | float | CPU load average for the prior fifteen-minute period. |
| quantum | int | Interval between metric collection for this metric entry. |
| disk_ro_rate | bigint | Disk read operations per second. |
| disk_wo_rate | bigint | Disk write operations per second. |
| disk_rb_rate | bigint | Bytes per second for disk read operations. |
| disk_wb_rate | bigint | Bytes per second for disk write operations. |
| net_rp_rate | bigint | Packets per second on the system network for read operations. |
| net_wp_rate | bigint | Packets per second on the system network for write operations. |

**Table C.1** gpperfmon.system_history

| column | type | description |
|---|---|---|
| net_rb_rate | bigint | Bytes per second on the system network for read operations. |
| net_wb_rate | bigint | Bytes per second on the system network for write operations. |

# database_now

The table *database_now* contains database-wide query information for current queries. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current database-wide query data is stored in *database_now* for each quantum period, after which is passes to *database_tail*..

**Table C.1** gpperfmon.iterators_history

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| queries_total | int | The total number of queries. |
| queries_running | int | The number of queries currently running. |
| queries_queued | int | The number of queries queued. |

# database_tail

The table *database_tail* contains database-wide query information for current queries. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current database-wide query data is stored in *database_tail* between flushing from *database_now* and automatic commitment to the *database_history* table..

**Table C.1** gpperfmon.iterators_history

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| queries_total | int | The total number of queries. |
| queries_running | int | The number of queries currently running. |
| queries_queued | int | The number of queries queued. |

# database_history

The table *database_history* contains historical database-wide query information. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need..

**Table C.1** gpperfmon.iterators_history

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| queries_total | int | The total number of queries. |
| queries_running | int | The number of queries running at data collection time for this row. |
| queries_queued | int | The number of queries queued at data collection time for this row. |

## queries_now

The table *queries_now* contains currently running query status information. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current query status data is stored in *quieries_now* until the query is completed, refreshing every quantum period. When the query is completed, the query data is passed into *queries_tail*.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the query data tables.

**Table C.1** gpperfmon.queries_now

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| tmid | int | A time identifier for a particular query. All records associated with the query will have the same `tmid`. |
| ssid | int | The session id as shown by `gp_session_id`. All records associated with the query will have the same `ssid`. |
| ccnt | in | The command number within this session as shown by `gp_command_count`. All records associated with the query will have the same `ccnt`. |
| uname | varchar(64) | Username that issued this query. |
| db | varchar(64) | Name of the database queried. |
| tsubmit | timestamp | Time the query was submitted. |
| tstart | timestamp | Time the query was started. |
| tfin | timestamp | Time the query finished. For running queries, this value is always zero. |
| status | varchar(64) | Status of the query -- `start`, `done`, or `abort`. |
| rows_out | bigin | Rows out for the query. |
| cpu_elapsed | bigin | CPU usage for this query across all segments. This is the sum of the CPU usage values taken from all segments in the database array. |
| cpu_currpct | float | Current CPU percent average for all processes executing this query. The percentages for all processes running on each segment are averaged, and then the average of all those values is calculated to render this metric. |
| skew_cpu | float | Coefficient of variation for `cpu_elapsed` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95. |
| skew_rows | float | Coefficient of variation for `rows_in` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95. |
| query_hash | bigint | This column is not implemented at time of writing. |
| query_text | text | Query text for this query. |
| query_plan | text | This column is not implemented at time of writing. |

## queries_tail

The table *queries_tail* contains currently running query status information. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/` . Current query status data is stored in *queries_tail* between flushing from *queries_now* and automatic commitment to the *queries_history* table.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the query data tables.

**Table C.1** gpmon_2.queries_now

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| tmid | int | A time identifier for a particular query. All records associated with the query will have the same `tmid`. |
| ssid | int | The session id as shown by `gp_session_id`. All records associated with the query will have the same `ssid`. |
| ccnt | in | The command number within this session as shown by `gp_command_count`. All records associated with the query will have the same `ccnt`. |
| uname | varchar(64) | Username that issued this query. |
| db | varchar(64) | Name of the database queried. |
| tsubmit | timestamp | Time the query was submitted. |
| tstart | timestamp | Time the query was started. |
| tfin | timestamp | Time the query finished. For running queries, this value is always zero. |
| status | varchar(64) | Status of the query -- `start`, `done`, or `abort`. |
| rows_out | bigin | Rows out for the query. |
| cpu_elapsed | bigin | CPU usage for this query across all segments. This is the sum of the CPU usage values taken from all segments in the database array. |
| cpu_currpct | float | Current CPU percent average is always zero in queries_tail. |
| skew_cpu | float | Coefficient of variation for `cpu_elapsed` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95. |
| skew_rows | float | Coefficient of variation for `rows_in` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95. |
| query_hash | bigint | This column is not implemented at time of writing. |
| query_text | text | Query text for this query. |
| query_plan | text | This column is not implemented at time of writing. |

# queries_history

The table *queries_history* stores historical query list status. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the query data tables.

**Table C.1** gpperfmon.queries_history

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| tmid | int | A time identifier for a particular query. All records associated with the query will have the same `tmid`. |
| ssid | int | The session id as shown by `gp_session_id`. All records associated with the query will have the same `ssid`. |
| ccnt | in | The command number within this session as shown by `gp_command_count`. All records associated with the query will have the same `ccnt`. |
| username | varchar(64) | Username that issued this query. |
| db | varchar(64) | Name of the database queried. |
| tsubmit | timestamp | Time the query was submitted. |
| tstart | timestamp | Time the query was started. |
| tfinish | timestamp | Time the query finished. |
| status | varchar(64) | Status of the query -- `start`, `done`, or `abort`. |
| rows_out | bigin | Rows out for the query. |
| cpu_elapsed | bigin | CPU usage for this query across all segments. This is the sum of the CPU usage values taken from all segments in the database array. |
| cpu_currpct | float | Current CPU percent average is always zero in historical data. |
| skew_cpu | float | Coefficient of variation for `cpu_elapsed` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95. |
| skew_rows | float | Coefficient of variation for `rows_in` of iterators across all segments for this query, multiplied by 100. For example, a value of .95 is rendered as 95. |
| query_hash | bigint | This column is not implemented at time of writing. |
| query_text | text | Query text for this query. |
| query_plan | text | This column is not implemented at time of writing. |

## iterators_now

The table *iterators_now* contains current query iterators and their metrics. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/`. Current iterator data is stored in *iterators_now* until the query is completed, refreshing every quantum period. When the query is completed, the detailed iterator data is passed into *iterators_tail*.

A query iterator refers to a node or operation in a query plan. For example, a sequential scan operation may be one node or iterator in a particular query plan. By looking at the metrics of the individual operations in a query plan, you can determine the status of a query's execution.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the iterator data tables.

**Table C.1** gpperfmon.iterators_now

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| tmid | int | A time identifier for a particular query. All iterator records associated with the query will have the same `tmid`. |
| ssid | int | The session id as shown by `gp_session_id`. All iterator records associated with the query will have the same `ssid`. |
| ccnt | in | The command number within this session as shown by `gp_command_count`. All iterator records associated with the query will have the same `ccnt`. |
| segid | int | The segment ID (`dbid` from `gp_configuration`). |
| pid | int | The process ID of the `postgres` process executing this iterator. |
| nid | int | The ID of this query plan node from the slice plan. |
| pnid | int | The ID of the parent query plan node from the slice plan. |
| hostname | varchar(64) | Hostname for the system where the iterator is executed. |
| ntype | varchar(64) | The query plan node operation type. Possible values are: `Adaptive Join, Hash Aggregate, Aggregate, Group Aggregate, Append, Append-Only Scan, BitmapAnd, Bitmap Heap Scan, Bitmap Index Scan, BitmapOr, External Scan, Function Scan, Group, Hash, Hash Join, Index Scan, Limit, Materialize, Merge Join, Redistribute Motion, Broadcast Motion, Gather Motion, Nested Loop, Result, Seq Scan, SetOp, ShareInputScan, Sort, Subquery Scan, Tid Scan, Unique, Values Scan, Window.` |
| nstatus | varchar(64) | The status of this query plan node operation. Possible values are: `Initialize, Executing and Finished.` |
| tstart | timestamp | Start time for the iterator. |
| tduration | int | Duration of the execution. |
| pmemsize | bigint | Memory size from Greenplum Database memory allocation data. |

**Table C.1** gpperfmon.iterators_now

| column | type | description |
|---|---|---|
| pmemmax | bigint | Maximum memory from Greenplum Database memory allocation data. |
| memsize | bigint | Memory size from OS memory allocation data. |
| memresid | bigint | Resident memory, as opposed to shared or virtual memory. |
| memshare | bigint | Shared memory figure from OS memory allocation data. |
| cpu_elapsed | bigint | Total CPU usage of the process executing the iterator. |
| cpu_currpct | float | This value is always zero for historical iterators. |
| phase | varchar(64) | This column is not implemented at time of writing. |
| rowsout | bigint | Number of rows output by the iterator. |
| rowsout_est | bigint | The query planner's estimate of rows output by the iterator. |
| m0_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. For all operations, this metric name is `Rows In`. |
| m0_unit | varchar(64) | The unit of measure for this metric. For all operations (ntype), this unit of measure is `Rows`. |
| m0_val | bigint | The value of this metric. |
| m0_est | bigint | The estimated value of this metric. |
| m1_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Tuples`<br>`Aggregate - Agg Total Spill Tuples`<br>`Group Aggregate - Agg Total Spill Tuples`<br>`Append - Append Current Input Source`<br>`Append-Only Scan - Rescans`<br>`Bitmap Heap Scan - Bitmap Heap Scan Pages`<br>`Hash - Hash Spill Batches`<br>`Hash Join - Hash Join Spill Batches`<br>`Index Scan - Index Scan Restore Pos`<br>`Merge Join - Merge Join Inner Side Tuples`<br>`Nested Loop - Nested Loop Inner Tuples`<br>`Seq Scan - SeqScan Page Stats`<br>`ShareInputScan - ShareInputScan Restore Pos`<br>`Sort - Sort Memory Usage`<br>`Subquery Scan - Subquery Scan Rescan` |

**Table C.1** gpperfmon.iterators_now

| column | type | description |
|---|---|---|
| m1_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Tuples`<br>`Aggregate - Tuples`<br>`Group Aggregate - Tuples`<br>`Append - Inputs`<br>`Append-Only Scan - Rescans`<br>`Bitmap Heap Scan - Pages`<br>`Hash - Batches`<br>`Hash Join - Batches`<br>`Index Scan - Restores`<br>`Merge Join - Tuples`<br>`Nested Loop - Tuples`<br>`Seq Scan - Pages`<br>`ShareInputScan - Restores`<br>`Sort - Bytes`<br>`Subquery Scan - Rescans` |
| m1_val | bigint | The value of this metric. |
| m1_est | bigint | The estimated value of this metric. |
| m2_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Bytes`<br>`Aggregate - Agg Total Spill Bytes`<br>`Group Aggregate -Agg Total Spill Bytes`<br>`Hash - Hash Spill Tuples`<br>`Hash Join - Hash Join Spill Tuples`<br>`Index Scan - Index Scan Rescan`<br>`Merge Join - Merge Join Outer Side Tuples`<br>`Nested Loop - Nested Loop Outer Tuples`<br>`Seq Scan - SeqScan Restore Pos`<br>`ShareInputScan - ShareInputScan Rescan`<br>`Sort - Sort Spill Tuples` |

**Table C.1** gpperfmon.iterators_now

| column | type | description |
|---|---|---|
| m2_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Bytes`<br>`Aggregate - Bytes`<br>`Group Aggregate - Bytes`<br>`Hash - Tuples`<br>`Hash Join - Tuples`<br>`Index Scan - Rescans`<br>`Merge Join - Tuples`<br>`Nested Loop - Tuples`<br>`Seq Scan - Restores`<br>`ShareInputScan - Rescans`<br>`Sort - Tuples` |
| m2_val | bigint | The value of this metric. |
| m2_est | bigint | The estimated value of this metric. |
| m3_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Batches`<br>`Aggregate - Agg Total Spill Batches`<br>`Group Aggregate - Agg Total Spill Batches`<br>`Hash - Hash Spill Bytes`<br>`Hash Join - Hash Join Spill Bytes`<br>`Seq Scan - SeqScan Rescan`<br>`Sort - Sort Spill Bytes` |
| m3_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Batches`<br>`Aggregate - Batches`<br>`Group Aggregate - Batches`<br>`Hash - Bytes`<br>`Hash Join - Bytes`<br>`Seq Scan - Rescans`<br>`Sort - Bytes` |
| m3_val | bigint | The value of this metric. |
| m3_est | bigint | The estimated value of this metric. |

**Table C.1** gpperfmon.iterators_now

| column | type | description |
|---|---|---|
| m4_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Pass`<br>`Aggregate - Agg Total Spill Pass`<br>`Group Aggregate -Agg Total Spill Pass`<br>`Sort - Sort Spill Pass` |
| m4_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Passes`<br>`Aggregate - Passes`<br>`Group Aggregate - Passes`<br>`Sort - Passes` |
| m4_val | bigint | The value of this metric. |
| m4_est | bigint | The estimated value of this metric. |
| m5_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Current Spill Pass Read Tuples`<br>`Aggregate - Agg Current Spill Pass Read Tuples`<br>`Group Aggregate - Agg Current Spill Pass Read Tuples`<br>`Sort - Sort Current Spill Pass Tuples` |
| m5_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Tuples`<br>`Aggregate - Tuples`<br>`Group Aggregate - Tuples`<br>`Sort - Tuples` |
| m5_val | bigint | The value of this metric. |
| m5_est | bigint | The estimated value of this metric. |
| m6_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Current Spill Pass Read Bytes`<br>`Aggregate - Agg Current Spill Pass Read Bytes`<br>`Group Aggregate - Agg Current Spill Pass Read Bytes`<br>`Sort - Sort Current Spill Pass Bytes` |

**Table C.1** gpperfmon.iterators_now

| column | type | description |
|---|---|---|
| m6_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Bytes`<br>`Aggregate - Bytes`<br>`Group Aggregate - Bytes`<br>`Sort - Bytes` |
| m6_val | bigint | The value of this metric. |
| m6_est | bigint | The estimated value of this metric. |
| m7_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Agg Current Spill Pass Tuples`<br>`Aggregate - Agg Current Spill Pass Tuples`<br>`Group Aggregate - Agg Current Spill Pass Tuples` |
| m7_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Tuples`<br>`Aggregate - Tuples`<br>`Group Aggregate - Tuples` |
| m7_val | bigint | The value of this metric. |
| m7_est | bigint | The estimated value of this metric. |
| m8_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Agg Current Spill Pass Bytes`<br>`Aggregate - Agg Total Spill Pass Bytes`<br>`Group Aggregate -Agg Total Spill Pass Bytes` |
| m8_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Bytes`<br>`Aggregate - Bytes`<br>`Group Aggregate - Bytes` |
| m8_val | bigint | The value of this metric. |
| m8_est | bigint | The estimated value of this metric. |

**Table C.1** gpperfmon.iterators_now

| column | type | description |
|---|---|---|
| m9_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Agg Current Spill Pass Batch`<br>`Aggregate - Agg Total Spill Pass Batch`<br>`Group Aggregate -Agg Total Spill Pass Batch` |
| m9_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Batches`<br>`Aggregate - Batches`<br>`Group Aggregate - Batches` |
| m9_val | bigint | The value of this metric. |
| m9_est | bigint | The estimated value of this metric. |
| t0_name | varchar(64) | This column is a label for *t0_val*. Its value is always “Name.” |
| t0_val | varchar(128) | The name of the relation being scanned by an iterator. This metric is collected only for iterators that perform scan operations such as a sequential scan or function scan. |

# iterators_tail

The table *iterators_tail* contains current query iterators and their metrics. This external table resides in `$MASTER_DATA_DIRECTORY/gpperfmon/data/` . Current iterator data is stored in *iterators_tail* between flushing from *iterators_now* and automatic commitment to *iterators_history*.

A query iterator refers to a node or operation in a query plan. For example, a sequential scan operation may be one node or iterator in a particular query plan. By looking at the metrics of the individual operations in a query plan, you can determine the status of a query's execution.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the iterator data tables.

**Table C.1** gpmon_2.iterators_now

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| tmid | int | A time identifier for a particular query. All iterator records associated with the query will have the same `tmid`. |
| ssid | int | The session id as shown by `gp_session_id`. All iterator records associated with the query will have the same `ssid`. |
| ccnt | in | The command number within this session as shown by `gp_command_count`. All iterator records associated with the query will have the same `ccnt`. |
| segid | int | The segment ID (`dbid` from `gp_configuration`). |
| pid | int | The process ID of the `postgres` process executing this iterator. |
| nid | int | The ID of this query plan node from the slice plan. |
| pnid | int | The ID of the parent query plan node from the slice plan. |
| hostname | varchar(64) | Hostname for the system where the iterator is executed. |
| ntype | varchar(64) | The query plan node operation type. Possible values are: `Adaptive Join, Hash Aggregate, Aggregate, Group Aggregate, Append, Append-Only Scan, BitmapAnd, Bitmap Heap Scan, Bitmap Index Scan, BitmapOr, External Scan, Function Scan, Group, Hash, Hash Join, Index Scan, Limit, Materialize, Merge Join, Redistribute Motion, Broadcast Motion, Gather Motion, Nested Loop, Result, Seq Scan, SetOp, ShareInputScan, Sort, Subquery Scan, Tid Scan, Unique, Values Scan, Window.` |
| nstatus | varchar(64) | The status of this query plan node operation. Possible values are: `Initialize, Executing and Finished.` |
| tstart | timestamp | Start time for the iterator. |
| tduration | int | Duration of the execution. |
| pmemsize | bigint | Memory size from Greenplum Database memory allocation data. |

**Table C.1** gpmon_2.iterators_now

| column | type | description |
|---|---|---|
| pmemmax | bigint | Maximum memory from Greenplum Database memory allocation data. |
| memsize | bigint | Memory size from OS memory allocation data. |
| memresid | bigint | Resident memory, as opposed to shared or virtual memory. |
| memshare | bigint | Shared memory figure from OS memory allocation data. |
| cpu_elapsed | bigint | Total CPU usage of the process executing the iterator. |
| cpu_currpct | float | This value is always zero for historical iterators. |
| phase | varchar(64) | This column is not implemented at time of writing. |
| rowsout | bigint | Number of rows output by the iterator. |
| rowsout_est | bigint | The query planner's estimate of rows output by the iterator. |
| m0_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. For all operations, this metric name is `Rows In`. |
| m0_unit | varchar(64) | The unit of measure for this metric. For all operations (ntype), this unit of measure is `Rows`. |
| m0_val | bigint | The value of this metric. |
| m0_est | bigint | The estimated value of this metric. |
| m1_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Tuples`<br>`Aggregate - Agg Total Spill Tuples`<br>`Group Aggregate - Agg Total Spill Tuples`<br>`Append - Append Current Input Source`<br>`Append-Only Scan - Rescans`<br>`Bitmap Heap Scan - Bitmap Heap Scan Pages`<br>`Hash - Hash Spill Batches`<br>`Hash Join - Hash Join Spill Batches`<br>`Index Scan - Index Scan Restore Pos`<br>`Merge Join - Merge Join Inner Side Tuples`<br>`Nested Loop - Nested Loop Inner Tuples`<br>`Seq Scan - SeqScan Page Stats`<br>`ShareInputScan - ShareInputScan Restore Pos`<br>`Sort - Sort Memory Usage`<br>`Subquery Scan - Subquery Scan Rescan` |

**Table C.1** gpmon_2.iterators_now

| column | type | description |
|---|---|---|
| m1_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Tuples`<br>`Aggregate - Tuples`<br>`Group Aggregate - Tuples`<br>`Append - Inputs`<br>`Append-Only Scan - Rescans`<br>`Bitmap Heap Scan - Pages`<br>`Hash - Batches`<br>`Hash Join - Batches`<br>`Index Scan - Restores`<br>`Merge Join - Tuples`<br>`Nested Loop - Tuples`<br>`Seq Scan - Pages`<br>`ShareInputScan - Restores`<br>`Sort - Bytes`<br>`Subquery Scan - Rescans` |
| m1_val | bigint | The value of this metric. |
| m1_est | bigint | The estimated value of this metric. |
| m2_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Bytes`<br>`Aggregate - Agg Total Spill Bytes`<br>`Group Aggregate -Agg Total Spill Bytes`<br>`Hash - Hash Spill Tuples`<br>`Hash Join - Hash Join Spill Tuples`<br>`Index Scan - Index Scan Rescan`<br>`Merge Join - Merge Join Outer Side Tuples`<br>`Nested Loop - Nested Loop Outer Tuples`<br>`Seq Scan - SeqScan Restore Pos`<br>`ShareInputScan - ShareInputScan Rescan`<br>`Sort - Sort Spill Tuples` |

**Table C.1** gpmon_2.iterators_now

| column | type | description |
|---|---|---|
| m2_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Bytes`<br>`Aggregate - Bytes`<br>`Group Aggregate - Bytes`<br>`Hash - Tuples`<br>`Hash Join - Tuples`<br>`Index Scan - Rescans`<br>`Merge Join - Tuples`<br>`Nested Loop - Tuples`<br>`Seq Scan - Restores`<br>`ShareInputScan - Rescans`<br>`Sort - Tuples` |
| m2_val | bigint | The value of this metric. |
| m2_est | bigint | The estimated value of this metric. |
| m3_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Batches`<br>`Aggregate - Agg Total Spill Batches`<br>`Group Aggregate - Agg Total Spill Batches`<br>`Hash - Hash Spill Bytes`<br>`Hash Join - Hash Join Spill Bytes`<br>`Seq Scan - SeqScan Rescan`<br>`Sort - Sort Spill Bytes` |
| m3_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Batches`<br>`Aggregate - Batches`<br>`Group Aggregate - Batches`<br>`Hash - Bytes`<br>`Hash Join - Bytes`<br>`Seq Scan - Rescans`<br>`Sort - Bytes` |
| m3_val | bigint | The value of this metric. |
| m3_est | bigint | The estimated value of this metric. |

**Table C.1** gpmon_2.iterators_now

| column | type | description |
|---|---|---|
| m4_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Pass`<br>`Aggregate - Agg Total Spill Pass`<br>`Group Aggregate -Agg Total Spill Pass`<br>`Sort - Sort Spill Pass` |
| m4_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Passes`<br>`Aggregate - Passes`<br>`Group Aggregate - Passes`<br>`Sort - Passes` |
| m4_val | bigint | The value of this metric. |
| m4_est | bigint | The estimated value of this metric. |
| m5_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Current Spill Pass Read Tuples`<br>`Aggregate - Agg Current Spill Pass Read Tuples`<br>`Group Aggregate - Agg Current Spill Pass Read Tuples`<br>`Sort - Sort Current Spill Pass Tuples` |
| m5_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Tuples`<br>`Aggregate - Tuples`<br>`Group Aggregate - Tuples`<br>`Sort - Tuples` |
| m5_val | bigint | The value of this metric. |
| m5_est | bigint | The estimated value of this metric. |
| m6_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Current Spill Pass Read Bytes`<br>`Aggregate - Agg Current Spill Pass Read Bytes`<br>`Group Aggregate - Agg Current Spill Pass Read Bytes`<br>`Sort - Sort Current Spill Pass Bytes` |

**Table C.1** gpmon_2.iterators_now

| column | type | description |
|---|---|---|
| m6_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Bytes`<br>`Aggregate - Bytes`<br>`Group Aggregate - Bytes`<br>`Sort - Bytes` |
| m6_val | bigint | The value of this metric. |
| m6_est | bigint | The estimated value of this metric. |
| m7_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Agg Current Spill Pass Tuples`<br>`Aggregate - Agg Current Spill Pass Tuples`<br>`Group Aggregate - Agg Current Spill Pass Tuples` |
| m7_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Tuples`<br>`Aggregate - Tuples`<br>`Group Aggregate - Tuples` |
| m7_val | bigint | The value of this metric. |
| m7_est | bigint | The estimated value of this metric. |
| m8_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Agg Current Spill Pass Bytes`<br>`Aggregate - Agg Total Spill Pass Bytes`<br>`Group Aggregate -Agg Total Spill Pass Bytes` |
| m8_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Bytes`<br>`Aggregate - Bytes`<br>`Group Aggregate - Bytes` |
| m8_val | bigint | The value of this metric. |
| m8_est | bigint | The estimated value of this metric. |

**Table C.1** gpmon_2.iterators_now

| column | type | description |
|---|---|---|
| m9_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Agg Current Spill Pass Batch`<br>`Aggregate - Agg Total Spill Pass Batch`<br>`Group Aggregate -Agg Total Spill Pass Batch` |
| m9_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Batches`<br>`Aggregate - Batches`<br>`Group Aggregate - Batches` |
| m9_val | bigint | The value of this metric. |
| m9_est | bigint | The estimated value of this metric. |
| t0_name | varchar(64) | This column is a label for *t0_val*. Its value is always “Name.” |
| t0_val | varchar(128) | The name of the relation being scanned by an iterator. This metric is collected only for iterators that perform scan operations such as a sequential scan or function scan. |

# iterators_history

The table *iterators_history* contains historical query iterators and their metrics. Like other history tables in the monitor database, it is partitioned by month, allowing you to drop partitions for data that you no longer need.

A query iterator refers to a node or operation in a query plan. For example, a sequential scan operation may be one node or iterator in a particular query plan.

These *tmid*, *ssid* and *ccnt* columns are the composite key that uniquely identifies a query. These columns can be used to join the iterator data tables.

**Table C.1** gpperfmon.iterators_history

| column | type | description |
|---|---|---|
| ctime | timestamp | Time this row was created. |
| tmid | int | A time identifier for a particular query. All iterator records associated with the query will have the same `tmid`. |
| ssid | int | The session id as shown by `gp_session_id`. All iterator records associated with the query will have the same `ssid`. |
| ccnt | in | The command number within this session as shown by `gp_command_count`. All iterator records associated with the query will have the same `ccnt`. |
| segid | int | The segment ID (`dbid` from `gp_configuration`). |
| pid | int | The process ID of the `postgres` process executing this iterator. |
| nid | int | The ID of this query plan node from the slice plan. |
| pnid | int | The ID of the parent query plan node from the slice plan. |
| hostname | varchar(64) | Hostname for the system where the iterator is executed. |
| ntype | varchar(64) | The query plan node operation type. Possible values are: `Adaptive Join, Hash Aggregate, Aggregate, Group Aggregate, Append, Append-Only Scan, BitmapAnd, Bitmap Heap Scan, Bitmap Index Scan, BitmapOr, External Scan, Function Scan, Group, Hash, Hash Join, Index Scan, Limit, Materialize, Merge Join, Redistribute Motion, Broadcast Motion, Gather Motion, Nested Loop, Result, Seq Scan, SetOp, ShareInputScan, Sort, Subquery Scan, Tid Scan, Unique, Values Scan, Window.` |
| nstatus | varchar(64) | The status of this query plan node operation. Possible values are: `Initialize, Executing and Finished.` |
| tstart | timestamp | Start time for the iterator. |
| tduration | int | Duration of the execution. |
| pmemsize | bigint | Memory size from Greenplum Database memory allocation data. |
| pmemmax | bigint | Maximum memory from Greenplum Database memory allocation data. |

**Table C.1** gpperfmon.iterators_history

| column | type | description |
|---|---|---|
| memsize | bigint | Memory size from OS memory allocation data. |
| memresid | bigint | Resident memory, as opposed to shared or virtual memory. |
| memshare | bigint | Shared memory figure from OS memory allocation data. |
| cpu_elapsed | bigint | Total CPU usage of the process executing the iterator. |
| cpu_currpct | float | This value is always zero for historical iterators. |
| phase | varchar(64) | This column is not implemented at time of writing. |
| rowsout | bigint | Number of rows output by the iterator. |
| rowsout_est | bigint | The query planner's estimate of rows output by the iterator. |
| m0_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. For all operations, this metric name is `Rows In`. |
| m0_unit | varchar(64) | The unit of measure for this metric. For all operations (ntype), this unit of measure is `Rows`. |
| m0_val | bigint | The value of this metric. |
| m0_est | bigint | The estimated value of this metric. |
| m1_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Tuples`<br>`Aggregate - Agg Total Spill Tuples`<br>`Group Aggregate - Agg Total Spill Tuples`<br>`Append - Append Current Input Source`<br>`Append-Only Scan - Rescans`<br>`Bitmap Heap Scan - Bitmap Heap Scan Pages`<br>`Hash - Hash Spill Batches`<br>`Hash Join - Hash Join Spill Batches`<br>`Index Scan - Index Scan Restore Pos`<br>`Merge Join - Merge Join Inner Side Tuples`<br>`Nested Loop - Nested Loop Inner Tuples`<br>`Seq Scan - SeqScan Page Stats`<br>`ShareInputScan - ShareInputScan Restore Pos`<br>`Sort - Sort Memory Usage`<br>`Subquery Scan - Subquery Scan Rescan` |

**Table C.1** gpperfmon.iterators_history

| column | type | description |
|---|---|---|
| m1_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Tuples`<br>`Aggregate - Tuples`<br>`Group Aggregate - Tuples`<br>`Append - Inputs`<br>`Append-Only Scan - Rescans`<br>`Bitmap Heap Scan - Pages`<br>`Hash - Batches`<br>`Hash Join - Batches`<br>`Index Scan - Restores`<br>`Merge Join - Tuples`<br>`Nested Loop - Tuples`<br>`Seq Scan - Pages`<br>`ShareInputScan - Restores`<br>`Sort - Bytes`<br>`Subquery Scan - Rescans` |
| m1_val | bigint | The value of this metric. |
| m1_est | bigint | The estimated value of this metric. |
| m2_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Bytes`<br>`Aggregate - Agg Total Spill Bytes`<br>`Group Aggregate -Agg Total Spill Bytes`<br>`Hash - Hash Spill Tuples`<br>`Hash Join - Hash Join Spill Tuples`<br>`Index Scan - Index Scan Rescan`<br>`Merge Join - Merge Join Outer Side Tuples`<br>`Nested Loop - Nested Loop Outer Tuples`<br>`Seq Scan - SeqScan Restore Pos`<br>`ShareInputScan - ShareInputScan Rescan`<br>`Sort - Sort Spill Tuples` |

**Table C.1** gpperfmon.iterators_history

| column | type | description |
|---|---|---|
| m2_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Bytes`<br>`Aggregate - Bytes`<br>`Group Aggregate - Bytes`<br>`Hash - Tuples`<br>`Hash Join - Tuples`<br>`Index Scan - Rescans`<br>`Merge Join - Tuples`<br>`Nested Loop - Tuples`<br>`Seq Scan - Restores`<br>`ShareInputScan - Rescans`<br>`Sort - Tuples` |
| m2_val | bigint | The value of this metric. |
| m2_est | bigint | The estimated value of this metric. |
| m3_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Batches`<br>`Aggregate - Agg Total Spill Batches`<br>`Group Aggregate - Agg Total Spill Batches`<br>`Hash - Hash Spill Bytes`<br>`Hash Join - Hash Join Spill Bytes`<br>`Seq Scan - SeqScan Rescan`<br>`Sort - Sort Spill Bytes` |
| m3_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Batches`<br>`Aggregate - Batches`<br>`Group Aggregate - Batches`<br>`Hash - Bytes`<br>`Hash Join - Bytes`<br>`Seq Scan - Rescans`<br>`Sort - Bytes` |
| m3_val | bigint | The value of this metric. |
| m3_est | bigint | The estimated value of this metric. |

**Table C.1** gpperfmon.iterators_history

| column | type | description |
|---|---|---|
| m4_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Total Spill Pass`<br>`Aggregate - Agg Total Spill Pass`<br>`Group Aggregate -Agg Total Spill Pass`<br>`Sort - Sort Spill Pass` |
| m4_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Passes`<br>`Aggregate - Passes`<br>`Group Aggregate - Passes`<br>`Sort - Passes` |
| m4_val | bigint | The value of this metric. |
| m4_est | bigint | The estimated value of this metric. |
| m5_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Current Spill Pass Read Tuples`<br>`Aggregate - Agg Current Spill Pass Read Tuples`<br>`Group Aggregate - Agg Current Spill Pass Read Tuples`<br>`Sort - Sort Current Spill Pass Tuples` |
| m5_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Tuples`<br>`Aggregate - Tuples`<br>`Group Aggregate - Tuples`<br>`Sort - Tuples` |
| m5_val | bigint | The value of this metric. |
| m5_est | bigint | The estimated value of this metric. |
| m6_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Agg Current Spill Pass Read Bytes`<br>`Aggregate - Agg Current Spill Pass Read Bytes`<br>`Group Aggregate - Agg Current Spill Pass Read Bytes`<br>`Sort - Sort Current Spill Pass Bytes` |

**Table C.1** gpperfmon.iterators_history

| column | type | description |
|---|---|---|
| m6_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operations:<br>`Hash Aggregate - Bytes`<br>`Aggregate - Bytes`<br>`Group Aggregate - Bytes`<br>`Sort - Bytes` |
| m6_val | bigint | The value of this metric. |
| m6_est | bigint | The estimated value of this metric. |
| m7_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Agg Current Spill Pass Tuples`<br>`Aggregate - Agg Current Spill Pass Tuples`<br>`Group Aggregate - Agg Current Spill Pass Tuples` |
| m7_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Tuples`<br>`Aggregate - Tuples`<br>`Group Aggregate - Tuples` |
| m7_val | bigint | The value of this metric. |
| m7_est | bigint | The estimated value of this metric. |
| m8_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Agg Current Spill Pass Bytes`<br>`Aggregate - Agg Total Spill Pass Bytes`<br>`Group Aggregate -Agg Total Spill Pass Bytes` |
| m8_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Bytes`<br>`Aggregate - Bytes`<br>`Group Aggregate - Bytes` |
| m8_val | bigint | The value of this metric. |
| m8_est | bigint | The estimated value of this metric. |

**Table C.1** gpperfmon.iterators_history

| column | type | description |
|---|---|---|
| m9_name | varchar(64) | Each operation in a query plan (ntype) has metrics associated with it. This metric name is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Agg Current Spill Pass Batch`<br>`Aggregate - Agg Total Spill Pass Batch`<br>`Group Aggregate -Agg Total Spill Pass Batch` |
| m9_unit | varchar(64) | The unit of measure for this metric. This unit is as follows for the corresponding plan node operation:<br>`Hash Aggregate - Batches`<br>`Aggregate - Batches`<br>`Group Aggregate - Batches` |
| m9_val | bigint | The value of this metric. |
| m9_est | bigint | The estimated value of this metric. |
| t0_name | varchar(64) | This column is a label for *t0_val*. Its value is always “Name.” |
| t0_val | varchar(128) | The name of the relation being scanned by an iterator. This metric is collected only for iterators that perform scan operations such as a sequential scan or function scan. |

# Index

-- noweb: 17

## Symbols

$GPPERFMONHOME: 11
_tail prefix: 7

## A

active queries: 26, 27
agent: 5
  gpmmon: 6
  gpsmon: 6
aggregating data
  query metrics: 30
  system metrics: 28, 29
API: 6
architecture: 5
authentication: 13, 18

## B

bashrc: 12, 24
bin directory: 11
browser support: 23

## C

collection interval: 34
components: 5, 12, 32
conf directory: 11
configuration: 10, 12, 32
  Greenplum Database: 9
  SSL: 21
configuration files: 11
connections
  database connections: 9
  Greenplum Performance Monitor console: 13, 35
  TCP: 6
console: 7, 13, 35
CPU usage: 26
current data
  queries: 27
current data tables: 35
current query data: 27
current system data: 25
  detail by metric: 26

## D

data analysis: 7
data collection interval: 34
data tables
  current data: 6
  external: 9
  historical: 7
  tail: 7
database
  monitor database: 6
date_trunc function: 29, 31
default installation path: 12
disable monitor: 15
disk usage: 26
DN: 13, 18
docs directory: 11
documentation: 11

## E

enable monitor: 10
excess data: 34
external tables: 9

## F

file system: 6, 7
file system access: 35
files, installer: 10

## G

gp_enable_gpperfmon: 10, 15
gp_external_enable_exec: 10
gpmmon: 12, 34
gpmmon.pid: 11
gpmon user: 9
gpperfmon
  database: 6
  directory: 11
  --setup: 12
  utility: 12
gpperfmon --setup: 17, 32
gpperfmon.conf: 12, 17, 32, 34
gpperfmon_path.sh: 11, 12
gpperfmon_port: 10
gpscp: 14
gpssh: 14, 15
graphical user interface: 7, 23
Greenplum Database configuration: 9
Greenplum Performance Monitor Console: 7
  browser support: 23
gtar: 14
GUI: 7, 23

## H

historical data: 7
  : 28
  iterators data: 34
  queries: 34
  query metrics: 30, 31
  system metrics: 28, 29
HTTP server: 12

## I

installation directory: 11
installation path: 10, 12, 14
installer file: 11
installing
  on master host: 10
  segment host: 14
interconnect: 5
iterators: 27, 34
iterators_history: 34

## J

JDBC: 6

## L

lib directory: 11
lighttpd: 8, 21, 23
lighttpd.conf: 12, 32
load: 26
local parameters: 9
log file size: 35
log_location: 35
logging: 11
logs directory: 11

## M

master agent: 12, 32, 34
memory: 26
middleware: 8
min_detailed_query_time: 18, 34
min_query_time: 34
monitor user: 9

## N

network: 5, 6
network usage: 26
node type: 28

## O

OBDC: 6

## P

partitioning: 7
path, installlation: 10
permissions: 12, 34
pg_hba.conf: 9
platform support: 11
port: 10
  web server: 13, 18
Postgresql functions
  avg: 31
  date_func: 29, 31
  max: 31
  min: 31
postgresql.conf: 9, 15
preinstallation: 9
process
  gpmmon: 6
  gpsmon: 6
process, agent: 5
psql: 6

## Q

quantum: 6, 34
queries: 27
  long-running: 34
queries_history: 34
query
  current data tables: 35
  plan details: 28
  runtime: 28
  searching for queries: 30
  watching queries: 28
query metrics
  historical: 30
query plan: 27
  node types: 28

## R

realtime metrics: 25
reconfiguring: 17
RedHat: 11
remote: 35
removing the monitor: 15
restart: 32, 34
role, gpmon: 9

## S

secure communications: 13, 18
segment agent: 32, 34
segment agent installation: 14
server.port: 21
setting local parameters: 9
setup: 17
signed certificate: 21
Solaris: 11
SQL views: 29, 31
  historical query metrics: 31
  historical system metrics: 29
SSL: 8, 13, 18
  ssl.engine: 21
  ssl.pemfile: 21
starting the monitor: 17, 32
status: 32, 35
stopping the monitor: 17, 32
superuser: 9
supported browsers: 23
supported platforms: 11
swap: 26
system
  data: 25
  metrics: 10, 28

    requirements: 9, 10
    resources: 9
    workload: 9, 25
system_history: 29


## T


tables
    current data: 6, 35
    external: 9
    external tables: 35
    historical data: 7
    historical iterator data: 34
    historical query data: 34
    system_history: 29
    tail: 7
tail tables: 7
TCP connections: 6
temporary files: 7
troubleshooting: 35
tuning the monitor: 34


## U


uninstalling: 15
updates: 34
user interface: 7, 23, 35


## V


verbose: 35
version: 32
views: 29, 31


## W


watching queries: 28
web API: 21
web server: 8, 12, 23
web server port: 13, 18, 21
web service: 8
    API: 23
    starting: 32
    stopping: 32
www directory: 11